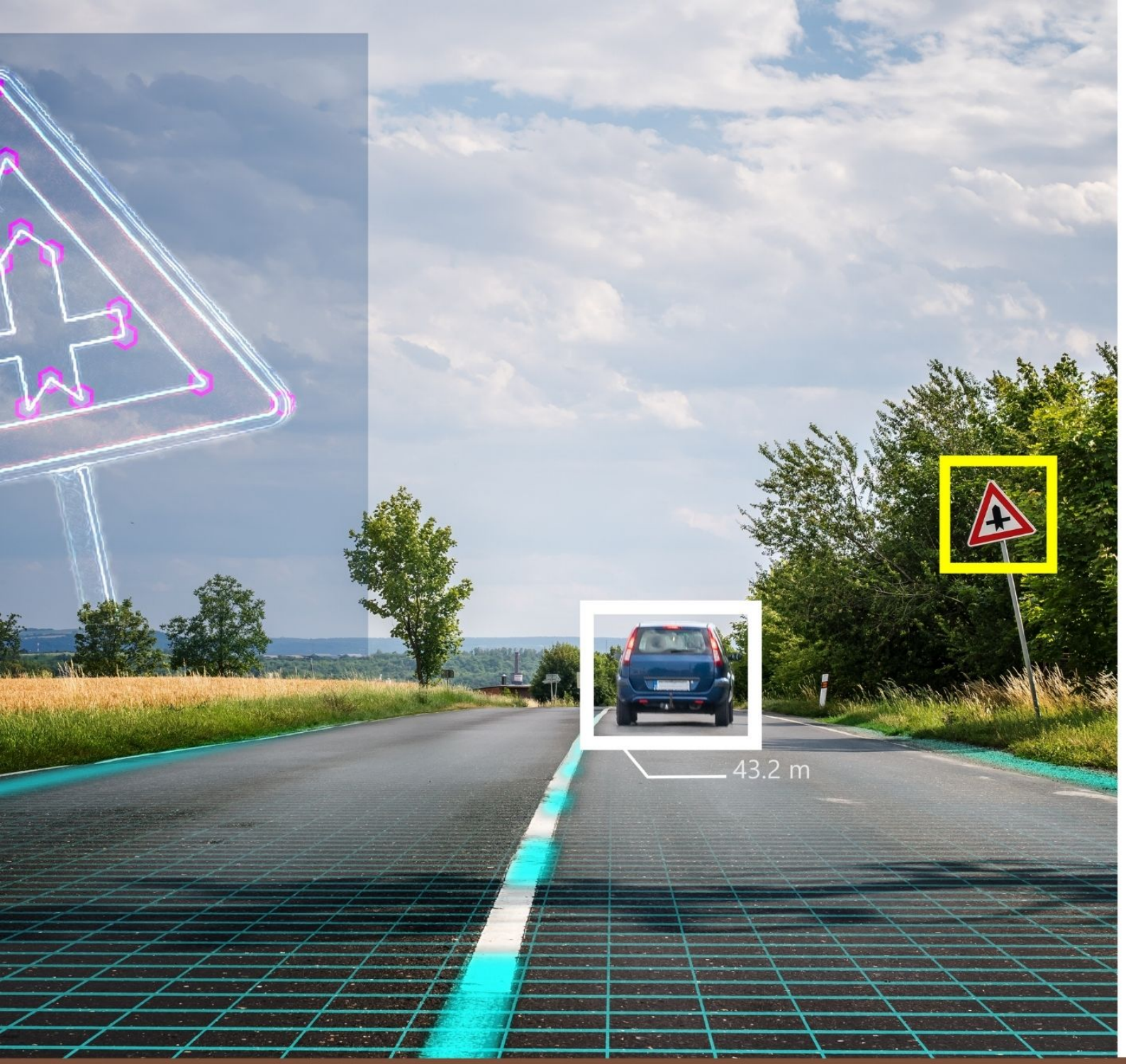


COMPUTER VISION

Prof. Jayanta Mukhopadhyay
Computer Science and Engineering
IIT Kharagpur

INDEX

S. No	Topic	Page No.
	Week 1	
1	Fundamentals of Image Processing Part I	1
2	Fundamentals of Imagr Processing Part II	21
3	Image Transform Part I	39
4	Image Transform Part II	58
	Week 2	
5	Projective Geometry â€“ Part I	79
6	Projective Geometry â€“ Part Ii	94
7	Projective Transformation	114
8	Homography:Properties â€“ Part I	140
9	Homography:Properties â€“ Part Ii	155
10	Homography:Properties â€“ Part Iii	172
	Week 3	
11	Camera Geometry â€“ Part I	189
12	Camera Geometry â€“ Part Ii	208
13	Camera Geometry â€“ Part Iii	223
14	Camera Geometry â€“ Part Iv	240
15	Camera Geometry â€“ Part V	261
	Week 4	
16	Stereo Geometry â€“ Part I	279
17	Stereo Geometry â€“ Part Ii	295
18	Stereo Geometry â€“ Part Iii	310
19	Stereo Geometry â€“ Part Iv	322
	Week 5	
20	Stereo Geometry â€“ Part Vi	334
21	Stereo Geometry â€“ Part V	346
22	Stereo Geometry â€“ Part Vii	363
23	Stereo Geometry â€“ Part Viii	380
	Week 6	
24	Feature Detection And Description â€“ Part I	394
25	Feature Detection And Description â€“ Part II	413
26	Feature Detection And Description â€“ Part III	432
27	Feature Detection And Description â€“ Part IV	449

28	Feature Detection And Description â€“ Part V	474
	Week 7	
29	Feature Matching And Model Fitting- Part I	485
30	Feature Matching And Model Fitting- Part Ii	502
31	Feature Matching And Model Fitting- Part Iii	514
32	Feature Matching And Model Fitting- Part Iv	527
33	Feature Matching And Model Fitting- Part V	540
	Week 8	
34	Color Fundamentals And Processing-Part I	558
35	Color Fundamentals And Processing-Part Ii	574
36	Color Fundamentals And Processing-Part Iii	591
37	Color Fundamentals And Processing-Part Iv	613
38	Color Fundamentals And Processing-Part V	633
39	Color Fundamentals And Processing-Part Vi	651
40	Color Fundamentals And Processing-Part Vii	665
	Week 9	
41	Range Image Processing â€“ Part I	679
42	Range Image Processing â€“ Part Ii	693
43	Range Image Processing â€“ Part Iii	707
44	Range Image Processing â€“ Part IV	720
45	Range Image Processing â€“ Part V	735
	Week 10	
46	Clustering And Classification Â€“ Part I	757
47	Clustering And Classification Â€“ Part Ii	780
48	Clustering And Classification Â€“ Part Iii	799
49	Clustering And Classification Â€“ Part Iv	816
50	Clustering And Classification Â€“ Part V	832
	Week 11	
51	Dimensional Reduction And Sparse Representation â€“ Part	855
52	Dimensional Reduction And Sparse Representation â€“ Part	871
53	Dimensional Reduction And Sparse Representation â€“ Part	891
54	Dimensional Reduction And Sparse Representation â€“ Part	909
	Week 12	
55	Deep Neural Architecture And Applications â€“ Part I	927
56	Deep Neural Architecture And Applications â€“ Part II	950
57	Deep Neural Architecture And Applications â€“ Part III	970

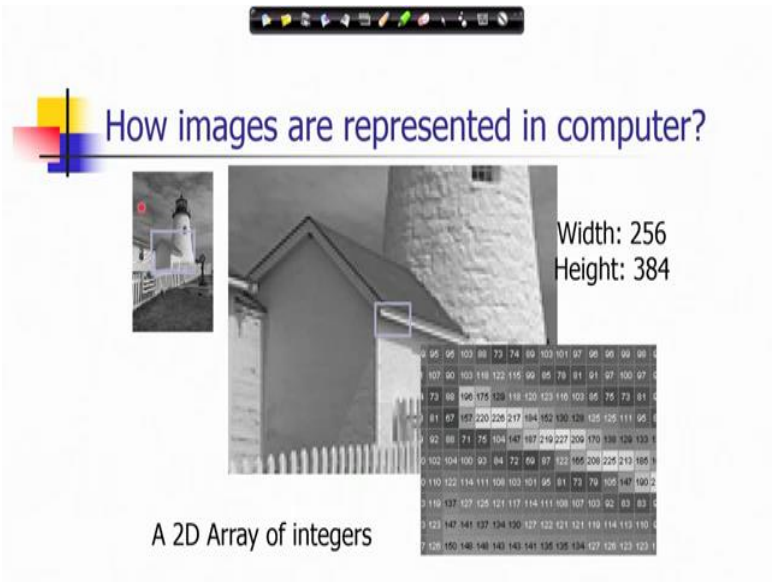
58	Deep Neural Architecture And Applications â€“ Part IV	988
59	Deep Neural Architecture And Applications â€“ Part V	1007
60	Deep Neural Architecture And Applications â€“ Part VI	1028

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 01
Fundamentals of Image Processing – Part I

In this lecture we will discuss about, different Fundamental operations of Image Processing and an overview of image processing.

(Refer Slide Time: 00:26)

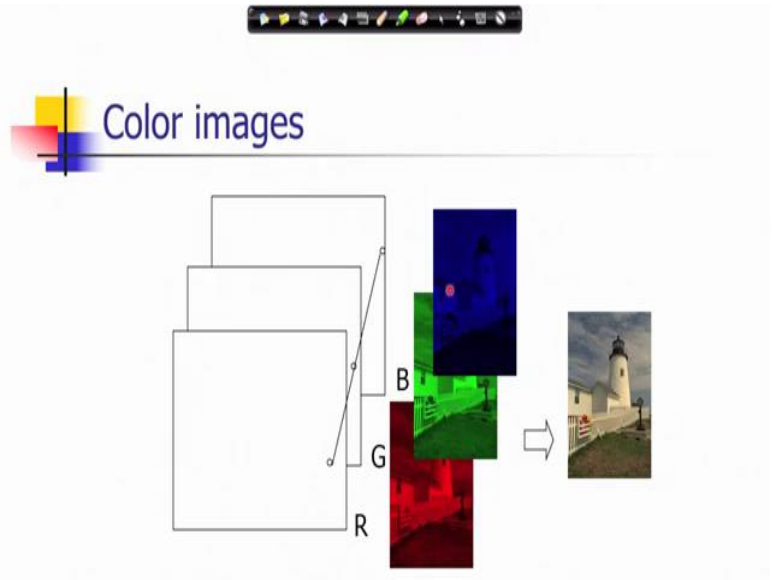


So, let us understand first how images are represented in computer. Let us consider an image which is displayed on your screen; and consider a small portion of the image as it is shown by this rectangle, if I zoom this portion then you will find the enlarging portions of the image. And you can see that some of the details are more visible here and, but there are certain kind of rectangular regions of the pixels regions which are also visible.

And, if I further zoom this area, then you will observe that further zooming those areas there are small squares of uniform illumination, those values are shown here by the numbers in those squared. So, finally, as you can see that, in the image at the very bottom level of representation, every point has a number and that number is representing the brightness value at that particular point. However, while displaying it on a screen it is a small area which is represented; which is representing that point and this brightness value is also shown at that point.

So, the image is represented as a 2D array of integers. So, these numbers are all those integers; and in a 2 dimensional array as you know you require to also mention the array size. So, those array size will be the width and height of the image for example, in this case for this image the width is 256, which means there are 256 points along its width and height is 384, there are 384 points along its height.

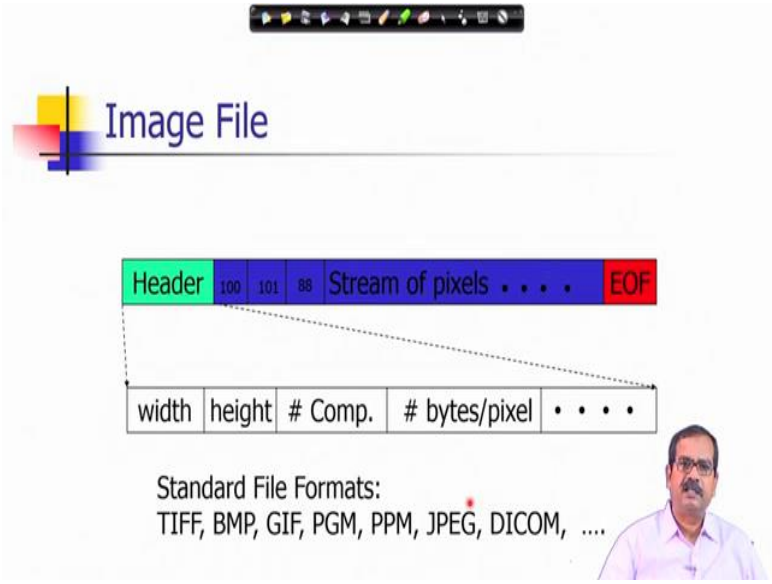
(Refer Slide Time: 02:26)



When we consider representing a colour image, there would be three such 2 dimensional arrays each for representing one of the primary colours like red, green and blue. If you consider any particular point of these array, all the respective locations all the respective array elements corresponding array elements with the same array indices, they will represent a combination of colour by this three primary colours. So, you will have a red component of the colour, if I can display only with the red colour in the screen.

And a green component of the image, which is displayed here by the green colour; and the blue component of the image which is displayed by blue colour. Now, once we super impose all these colours on a screen, then you can get the colour representation of the image itself. So, an image in this case is represented by three, 2 dimensional array.

(Refer Slide Time: 03:30)



And when this information is stored in a computer hard disk, in the secondary storage you know that any information in the secondary storage is stored as a file. Similarly, image has to be stored as a file and a file it consists of a stream of bytes. So, in this case every byte or a collection of byte will represent a pixels and we can consider that is as a stream of pixels.

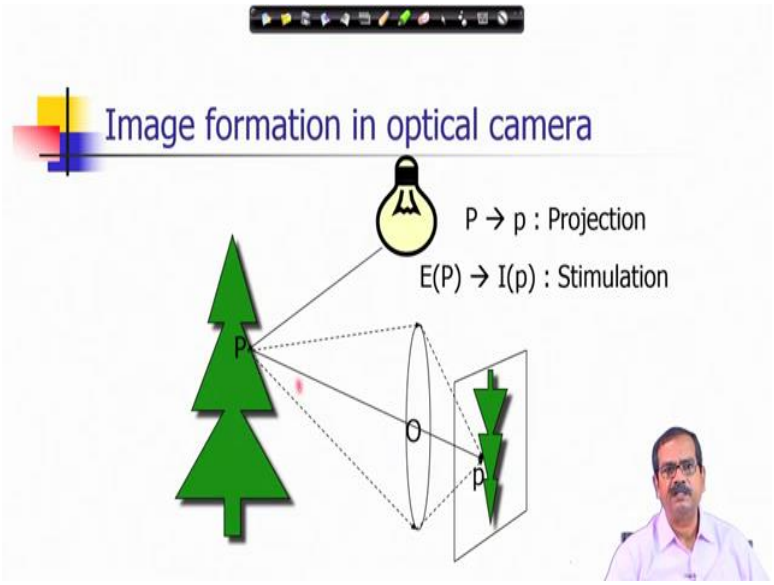
However, to represent the image in the in your program in your computation, you require other associated information's of the images of that image and which has to be stored also with this stream. Usually, they are stored a head of the stream in a very predefined format and which is called header of the image file. For example, a header can consist of this kind of information's like, it could it should contain the width information, height then number of components which means in this case for a colour channel it should be three for a colour image.

Number of bytes per pixel as I mentioned that, a pixel could be represented by number of bytes; in a very elementary representations usually it is 1 byte per pixel. And where the values it you know unsigned integer values vary from, 0 to 255 per pixel. of course, there should be an end of file representation for a file.

And there are different standard file formats which are available for an image representation, they are very much standardized and documents are available. So, when you get an image file in these formats, you should know the corresponding format and you

should parts the header and get the images. So, these are some example formats like TIFF, BMP, GIF etc.

(Refer Slide Time: 05:31)



So, let us consider, how an image is formed in an optical camera? Let us consider in a camera which is represented here, a lens of the camera which plays a very critical role. And, there is a plane where the image points are projected where the points of the 3 dimensional scene, they are projected on a 2 dimensional plane and that is actually the focal plane usually of these particular lens. And where the images are found and those points are finally, sensed by corresponding sensors and they are digitised and you get a digital representation.

So, if I consider that, how a point in a 3 dimensional space is mapped to an image point; so, let us see how it can take place, if you consider a point P where a light is reflected and that light is passing through the centre of the lens and intersecting at the plane of projections or the image plane. So, this point which is represented here by small p is a image of the scene point which is represented here by this capital P. So, image has been in this case image formation, has taken place due to the phenomena of reflection as you can see.

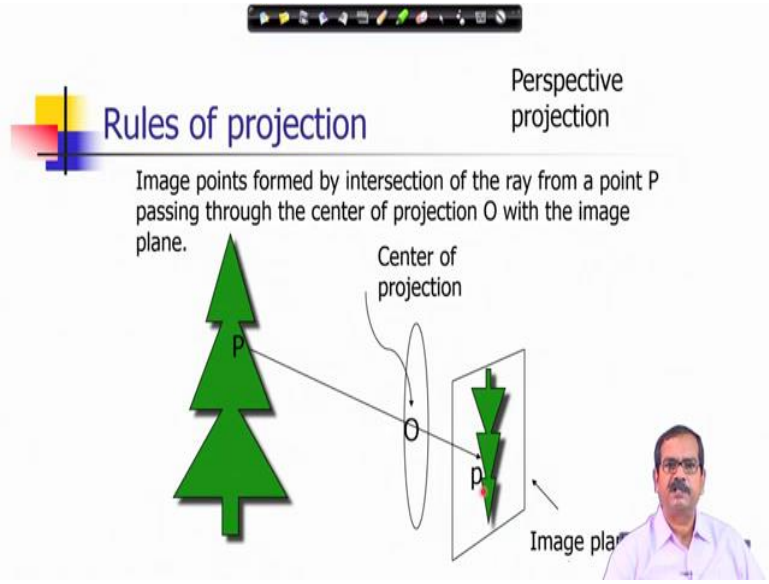
So, there is another information, which is also encoded at this image point that is a amount of reflection that amount of energy that is received at this point which is reflected from the point P. So, that is the action of lens, which takes care of this particular but it tries to get

as much as energy reflected energy tries to put them together on this plane p, collects it and focused it on this point p and that is what is called also focusing. And that is why you get a very sharp picture if it is properly focused, a sharp point representation of the same point.

So, this is another encoding. So, the amount of energy which is reflected from this point that should be received here and that is sensed. So, the interpretation of this image is that it keeps a brightness distribution in this 2 dimensional plane; where at each point you get the corresponding it is proportional to the amount of energy reflected from its corresponding same point.

Let us look at minutely that once again; what is a rule of projection that I mentioned here and that provides you a very simple mathematical tool to compute given a point P in the 3 dimensional world, what should be its corresponding image point in a 2 dimensional plane. So, as we can see in this case that, this can be formed if I draw a line from this point through a particular fixed point O which is here the centre of the lens and extend that ray which hits the corresponding image plane and that intersection point defines the image point of the 3 dimensional scene point.

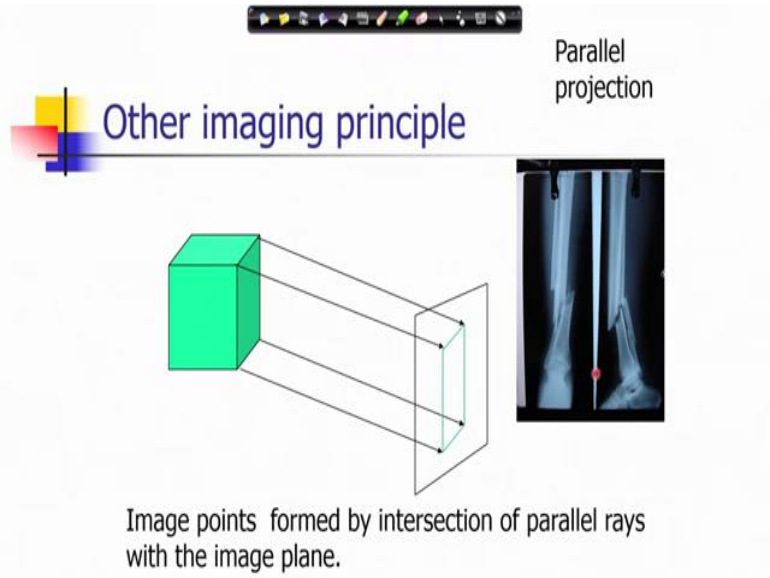
(Refer Slide Time: 08:53)



So, this is a centre of projection as I mentioned and this is the image plane. And so, we can summarize this rule as image points formed by intersection of the ray from a point P

and passing through the centre of projection O with the image plane. This kind of projection is known as perspective projection.

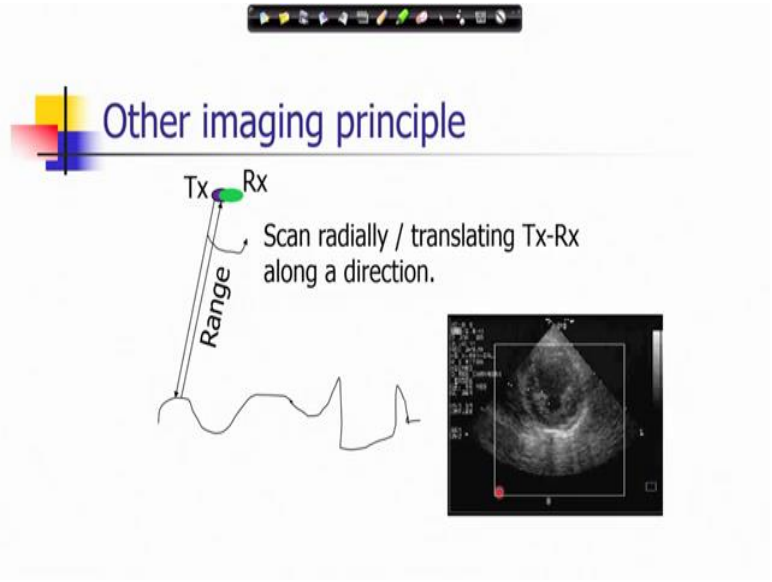
(Refer Slide Time: 09:14)



But, this is not the only way images are formed there could be other kinds of imaging principles other rules of projections can take place. For example, in this case what I am going to show here, there are image formed image of this cube where you know one of the planes of the cubes have been projected here. And, as you can see all these points are parallely projected to this plane, there is a particular direction which has to be considered in this case for this projection it could be normal to the plane, it could be any other directions in a 3 dimensional plane.

So, this rule we can summarize in this way that image points formed by intersection of parallel rays with the image plane. And, this kind of one example of this kind of imaging is X ray imaging where X ray beams parallel X ray beams, it passes through our know body through bones through tissues. And then it intersects the corresponding X ray plate, which acts likes a image plane in this case and forms the image and this projection is known as parallel projections.

(Refer Slide Time: 10:20)

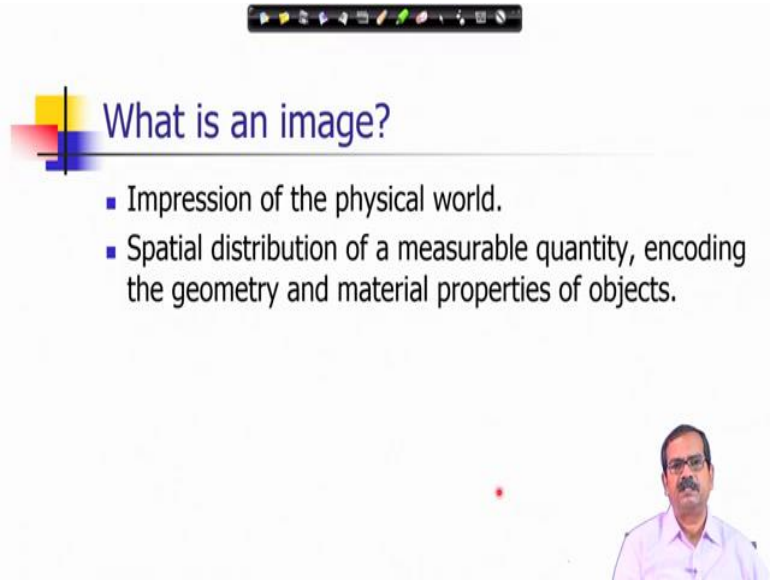


Let us take another imaging principle. Let us consider you have a surface of an object and your imaging sensor there is a transmitter, which transmits the transmits a electromagnetic wave or some acoustic wave and then, the reflected wave is received by receiver. So, the duration the time interval between transmission and reception that can be measured and if you know the velocity of the wave you can compute the corresponding distance from this point.

And consider it scan radially, you are taking this at every regular interval and you are scanning it radially over the surface points. And you can get or you can consider also, you can translate this transmitter receiver transmitter receiver along certain directions and performs its action repeatedly. So, for every surface point in that path you will get a distance. So, you can measure the distance not only that the amount of reflection, what you get from the surface that would also determine the orientation and surface property of this particular material.

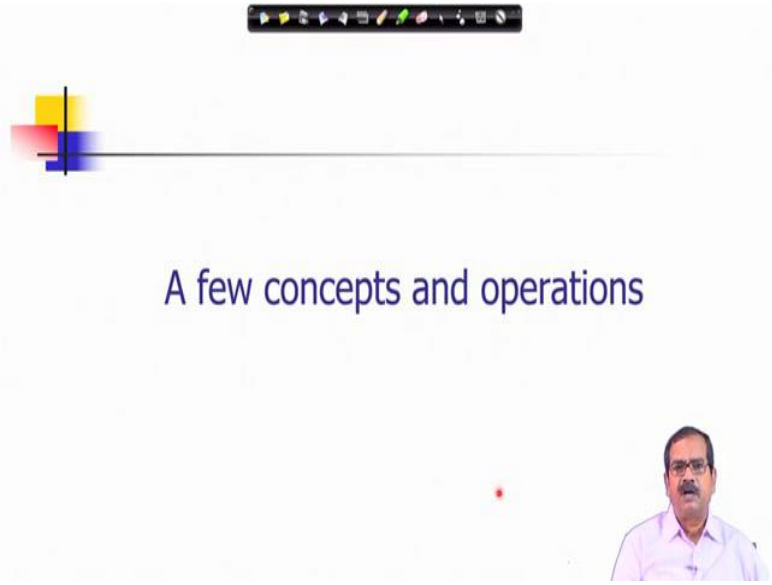
So, one example is this echocardiograms, where acoustic waves are used and ultra sound waves are used and so, this is one kind of example.

(Refer Slide Time: 11:43)



So, if I summarize that how what is an image, how do I define an image? It is in a very short sentence we can say, it is an impression of the physical world. And just to make it a little more just to elaborate it we can say that, it is a special distribution of a measurable quantity encoding the geometry and material properties of objects.

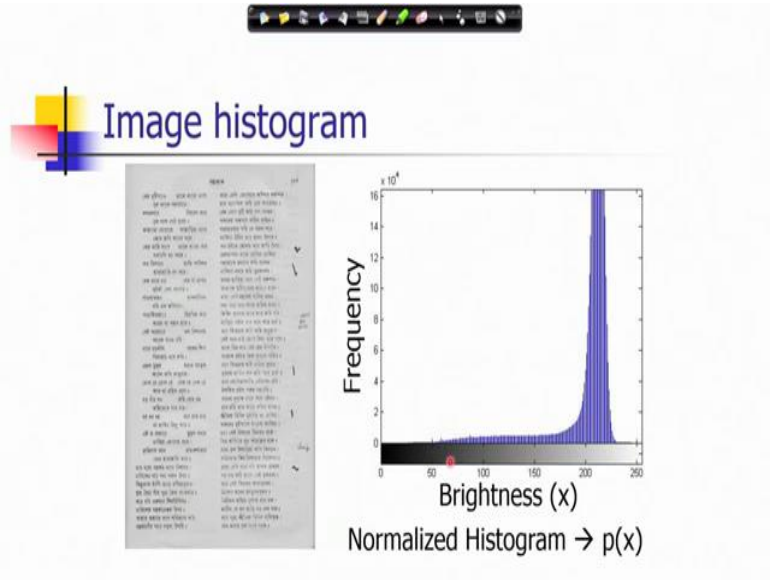
(Refer Slide Time: 12:14)



So, now I will be discussing a few concepts and operations, which are which are there in the image processing. And in this course we would require some of these concepts as I mentioned earlier that know you do not required to go through the first level image

processing course to attend this particular computer mission course, these are the primers that I will be discussing here. However, it would be better if you follow some image processing textbooks and know further know more details about this concepts.

(Refer Slide Time: 12:50)



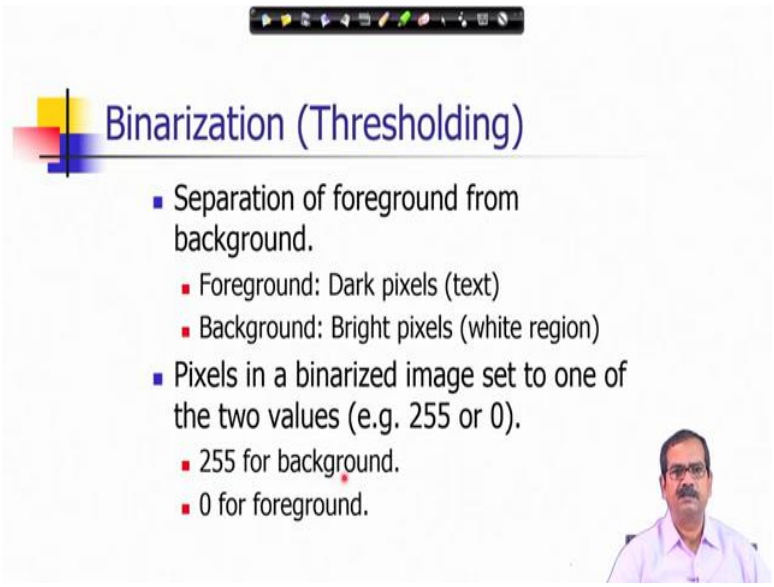
So, let us consider the first a very simple concept of images a very first level statistics of the distribution of this pixel values and, which can be captured in the form of a frequency distribution of the brightness values in this particular image. Here I have shown an image of a scanned page we can say this classes of images are document images. Once again in this image also you have those brightness values at every pixel.

And as you can see there are two types of pixels are there mostly, one is one is a text it depicts a text of the document the other one belongs to background. Usually in the histogram you should have found the this you should have obtained, a bimodal kind of characteristics. But in this case since you get so many white pixels, so it is more skewed and particularly the distribution in the text zone it looks little flat.

So, we will come to this point know how this could be processed further to make it more bimodal, but presently let us consider this is the let us concentrate on this fact that a. An image histogram is nothing, but the frequency distribution of this brightness value. And from this frequency distribution you can get the probability distribution you can convert it into a probability distribution of the brightness value.

If I normalize the histogram; that means, all this frequencies should be divided by the total number of pixels of the image, then you get the probability distribution of the brightness values.

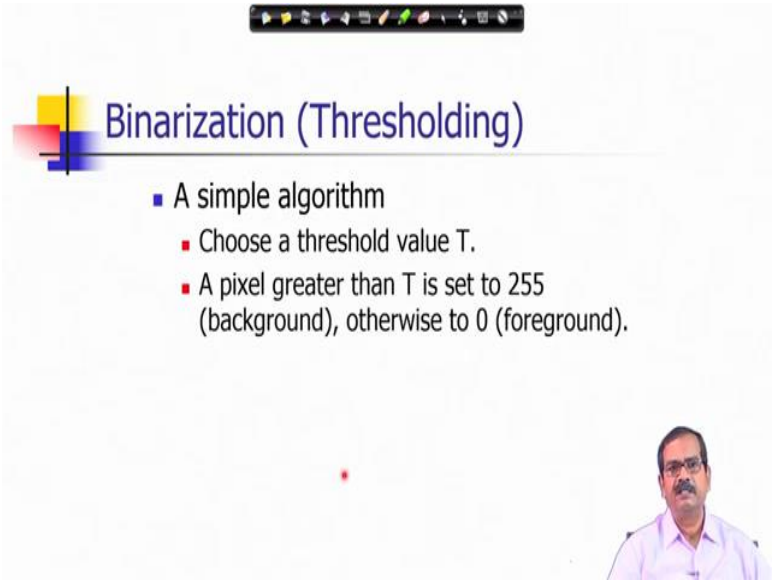
(Refer Slide Time: 14:29)



So, one of the problem of document image analysis is to separate the foreground from background and this process is called binarization process. So, and one of the simple technique of binarization is using a threshold value to declare that, whether a pixel is foreground or whether it is background ah.

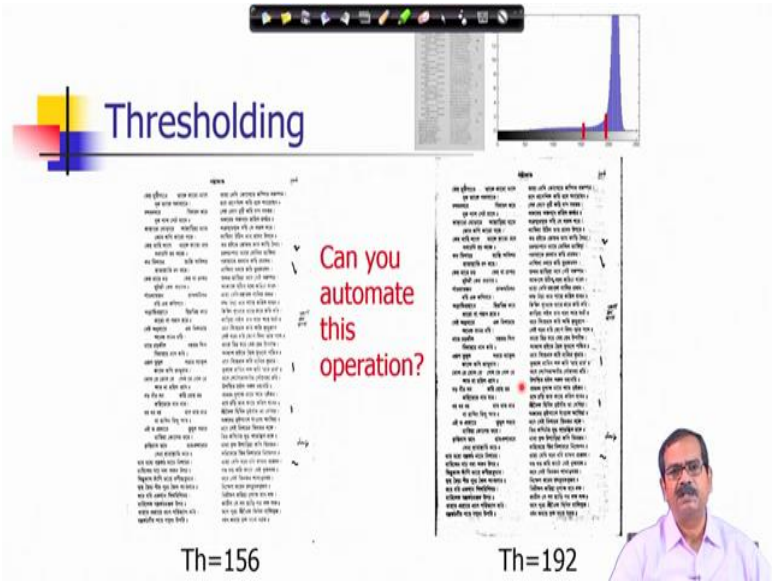
So, in our present context the example what I have given here foreground is the dark pixels and background is the bright pixels; so which are white region of the document. And pixels after binarization, it could be set to one of the two values for example, we can consider 255 represents the white region and 0 represents the text portions or dark pixels say this is what we can consider.

(Refer Slide Time: 15:30)



One of the simple algorithms at of this you know binarization could be as follows. Say, you can choose a threshold value T that is one of the brightness value in that intervals some value in the brightness interval. And a pixel greater than T is set to 255 otherwise it is set to 0.

(Refer Slide Time: 15:55)



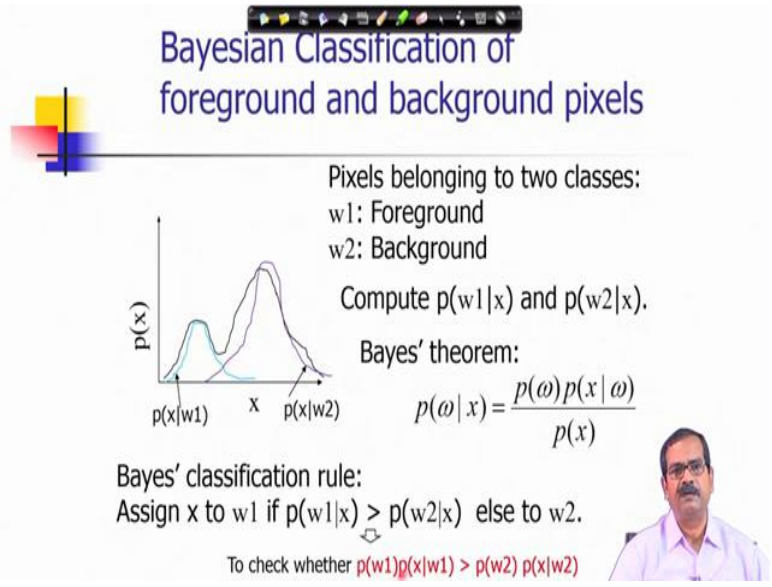
So, this is a very simple algorithm and let us see, what is the affect of this algorithm. Say, you consider that know this document and this is represented this is displaying the particular a histogram. Let us consider a particular value say 156 where you perform this

thresholding and then you get an image like this. So, you see that there are only two types of pixels, pixels with the value 0 and pixels with value 255 in this case when your threshold value is 156.

If I choose another value say 192 you get also another kind of binarized image; and you can see the difference between these two images. So, if the threshold value is higher than you get more foreground pixels your text becomes sharper here, but then there are more spurious noise in your in your document also which is not desirable. So, what is the optimum threshold value? What is a desirable threshold value which will make my text sharper which will look the foreground also sharper or they take the proper program pixels and also it should not contain the it should remove the noisy part of the pixels also.

So, this kind of manual choice of thresholding may not help when you are trying to process various documents. And one of the objective would be that to automate this operation of this thresholding ah.

(Refer Slide Time: 17:26)



So, one of the techniques, that I would be discussing here, is a Bayesian classification of foreground and background pixels of a particular image. In this case we consider that our histogram of the image is a bimodal histogram, a schematic diagram is shown here it is. Say, there are two modes, there are two peaks in this histogram and we our assumption is that most of the pixels which are around this mode around its particular peak, those pixels are they are coming from the foreground pixels.

And the and the pixels, which are coming from the background they are centered around this particular mode. So, we consider there are two classes of pixels and these are the know symbols of this class these are the notations of this class in this case say it has considered w 1 and w 2 just for the abstract representation of this problem.

So, what we need to do in this case? We need to compute the probability of a class w 1 given x,

$$p(w1|x)$$

and probability of a class w 2 given x,

$$p(w2|x)$$

So, this is because you know in Bayesian classification rule, we can assign the pixel x to class w1, if

$$p(w1|x) > p(w2|x)$$

otherwise we assign it to w2 that is a base Bayes classification rule. So, how we can compute this know this particular probability, which is called incidentally posterior probability. In this case, so we can apply Bayes theorem there and in the Bayes theorem you can see that in this case I have described the theorem.

So, consider this pixel x, so probability of a class given that pixel x, can be computed from this three quantities.

$$p(w|x) = \frac{p(w)p(x|w)}{p(x)}$$

And it is simpler to compute $p(x|w)$ than $p(w|x)$ directly because $p(x|w)$ is called likelihood. And we can assume that the pixels which are around, this part they form a distributions which are coming a class distribution which are coming from the foreground pixels and can assume they are Gaussian distribution.

Similarly, the pixels, so this is this is the probability distribution $p(x|w1)$ And similarly for the background class also we can consider another probability distributions, for the background class and that would be the probability distribution $p(x|w2)$. So, these

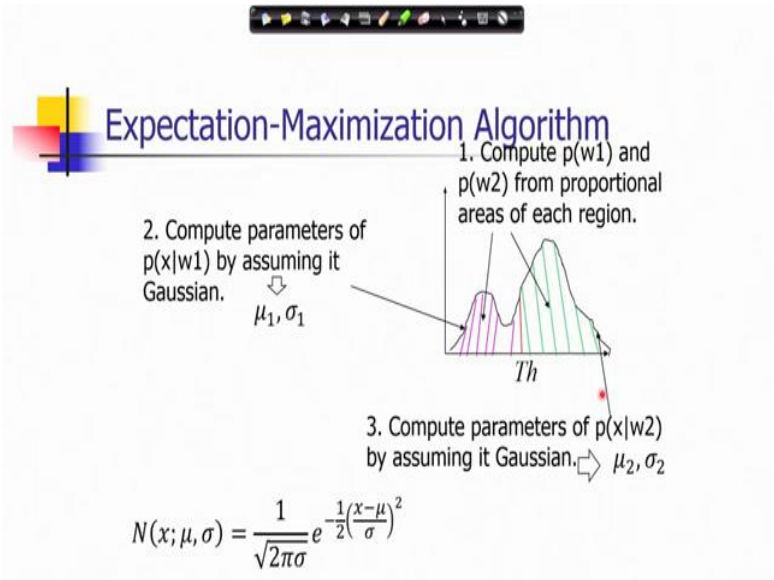
probabilities are called likelihood that could be easily that could be computed easily rather than computing this directly.

And also you can compute class probabilities $p(w)$, if I assume that know some threshold value is chosen then these proportional areas can give me those two probabilities. I will describe it in the subsequent slide, but what is interesting to note that actually you may not use $p(x)$ at all in this computations. Because after computing $p(w1|x)$ and $p(w2|x)$ you need to compare only

$$p(w1|x) > p(w2|x)$$

that would, because this is proportional to given a particular x, $p(x)$ is you know already given by the data itself.

(Refer Slide Time: 21:11)



So, let us see how these computations can be carried out? And there is a algorithm by which we determine this thresholds, we call this algorithms as expectation maximization algorithm. So, let me explain this algorithm here. So, let us consider that histogram once again histogram of the image or probability distribution of the pixels. And let us assume and a threshold value initially say at this point. So, this value divides the intervals this brightness interval into two halves.

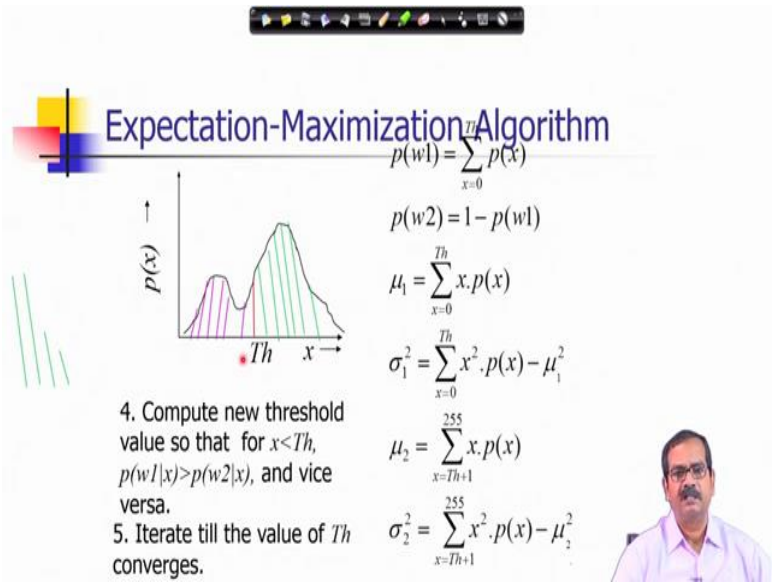
So, one we can consider this half belongs to say foreground region and this half belongs to a background region. And so, this is a representation for the foreground part and say

this is a representation of background part. So, what we can do? Given this threshold this threshold we can compute $p(w1)$ and $p(w2)$ why computing the areas area of this part and also areas of this part and take the proportional know areas of which region to compute that values.

So, this is how the probability of class probabilities could be computed, ones given a threshold value. And then, after that we can consider only concentrate only this values these classes only and from there we can compute the parameters of say $p(x|w1)$ by assuming it Gaussian. And similarly we can also compute parameters of $p(x|w2)$ by assuming it Gaussian.

So, if I relook at the Gaussian distributions function, it is a Gaussian distribution function as you can see there are two parameters; one is μ which is a mean of the distribution another one is σ which is the standard deviation of this distribution. So, what you need to do in this case to compute the probability this likelihood probability $p(x|w1)$, you just simply you need to compute this parameters then you can compute the probability of any value x given those parameters. So, let us assume that we call the parameters corresponding parameters for the class w 1 as μ1 and σ1 and corresponding parameters of w 2 as μ2 and σ2.

(Refer Slide Time: 23:34)



So, next what we will do that we will be considering the and this is how the corresponding parameters are computed in this case.

$$p(w1) = \sum_{x=0}^{Th} p(x)$$

$$p(w2) = 1 - p(w1)$$

$$\mu 1 = \sum_{x=0}^{Th} x.p(x)$$

$$\sigma_1^2 = \sum_{x=0}^{Th} x^2.p(x) - \mu_1^2$$

$$\mu 2 = \sum_{x=Th+1}^{255} x.p(x)$$

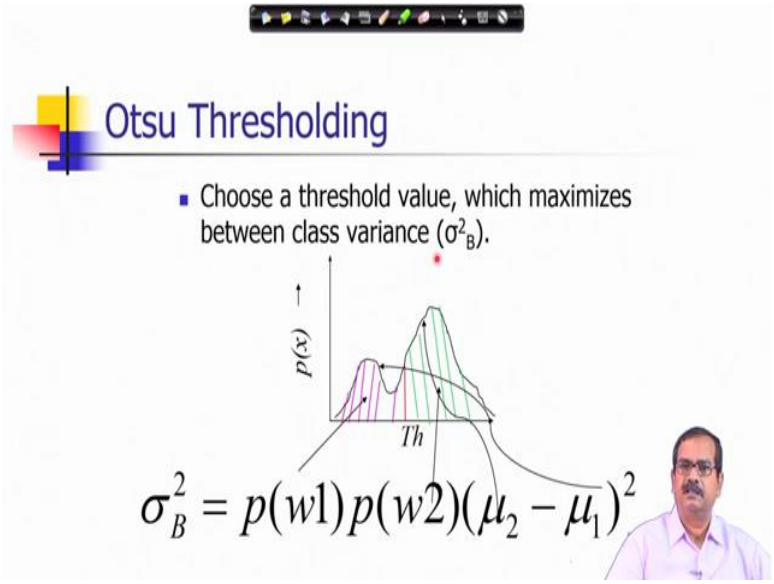
$$\sigma_2^2 = \sum_{x=Th+1}^{255} x^2.p(x) - \mu_2^2$$

So, we are computing the variances with this of this values and main of these values. And these are all simple mathematical arithmetic expressions of weighted means and weighted variances. So, if you look at the statistics, book of statistics it will be very clear how these values are computed by this expressions are there. So, once we get these values then we are determining a new threshold value such that, the

$$p(w1|x) > p(w2|x)$$

So, as soon as it becomes less, then we choose that threshold values, still choose that threshold value and that value would be a new value. So, we expected this should be a threshold value, but after computing this parameter after maximizing the probabilities of occurrences of these pixels then we found there is a better threshold value which will be giving in a bettered better probabilistic occurrences of this observations. So, we iterate this process, so that would be my new threshold value and we will be iterating this process till the process is converged. So, this is what is your Bayesian classification based binarization method.

(Refer Slide Time: 25:44)

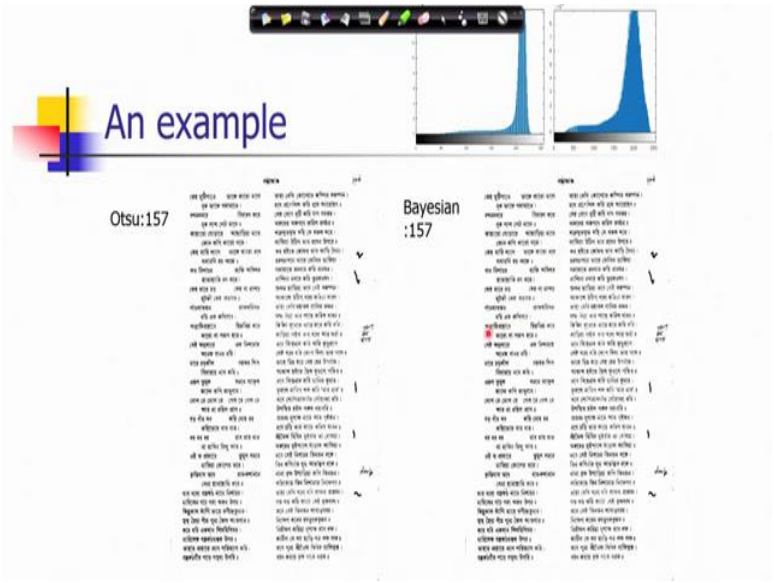


And there is another method also we can consider here, which is almost similar and which also defines an optimization function by which you can get the threshold value. So, this optimization function is the between class variance of the particular two classes. So, between class variances are also as you can see it is defined by those by those parameters which I have discussed earlier.

$$\sigma_B^2 = p(w1)p(w2)(\mu_2 - \mu_1)^2$$

So, given a threshold value I can compute this σ_B^2, like p(w1) from this part p(w2) from this part then μ_1 can be computed from here and μ_2 can be computed from here. So, you consider that you are computing thee value at every pixel value from in your intervals say 0 to 255 for example, and you are you consider the that pixel value where the between class variance is maximum and that has been that you consider as your threshold value. So, this thresholding principle this thresholding technique is proposed by Otsu and it is known as Otsu thresholding technique.

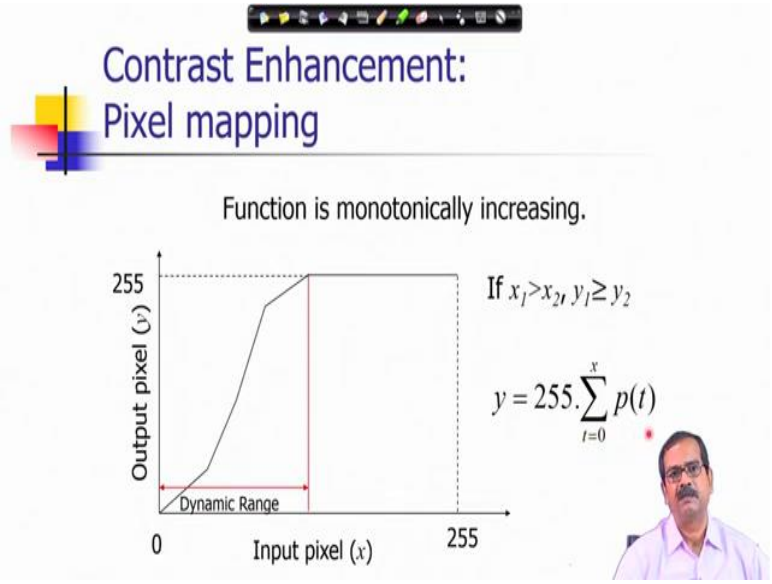
(Refer Slide Time: 27:03)



So, the example of these particular processing, you can see that with that particular document image. We have computed this Otsu thresholds and which is 157 in this case and it gives know this kind of image. And, if I consider the Bayesian thresholds, then we find an another image incidentally though the threshold values are same you can see that there is little bit of difference between these two images though quality of this two images are almost similar; this difference happens because we would like to make this histogram bimodal.

So, before applying Bayesian classification, we process the image so that, the histogram has a sharper modes also in the foreground zone. So, how we are processing it I will discuss in the you know next part.

(Refer Slide Time: 27:58)



So, in this case, so this is what is the method which I was referring at it is a contrast enhancement method and here the concept of pixel mapping is used. The pixel mapping concept is that you have an input pixel and which will be mapped to an output pixel in such a way that dynamic range of the input would be expanded. That means, suppose in this case dynamic range is from 0 to this value which could be say half of the interval approximately.

But in the output we are converting that dynamic range from 0 to 255 that makes the pixels sharper I mean contrast sharper. And one of the property of course, that you need to mention that you need to you know preserve that this function has to be monotonically increasing because, if you have two pixels x 2 and x 1

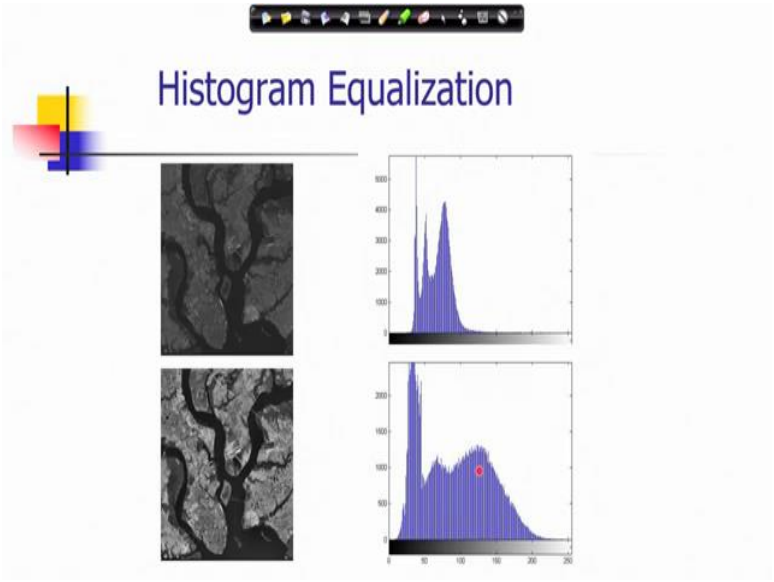
If $x_1 > x_2$, then $y_1 > y_2$

So, that keeps a consistency of displaying you know brighter pixel brighter and darker pixel darker, so that is why you require this property.

One of the popular function which is used in particular in this case, is this function from the probability distribution of the you know pixel itself. So, this is a cumulative distribution and

$$y = 255 \sum_{t=0}^{x} p(t)$$

(Refer Slide Time: 29:28)



So, if I do this operation we can see that, we get a contrast rate image where the features are more visible in this are more prominent here. You can also look at this histogram this histogram has similar shape, but the dynamic range has been expanded modes are more clearly visible. In fact, this is a technique what I was referring at this is a technique we applied in the document which has been processed for binarization. So, with this let me stop here, this is a first part of this particular talk, we will start further we will go for the next part in the next lecture.

Thank you very much for your listening.

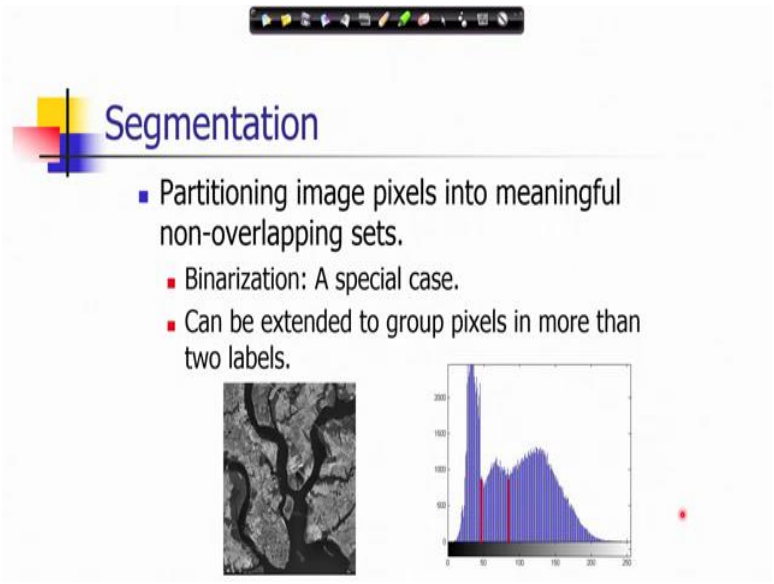
Keywords: Images, projection, histogram, thresholding, expectation maximization, Bayesian, equalization

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 02
Fundamentals of Image Processing – Part II

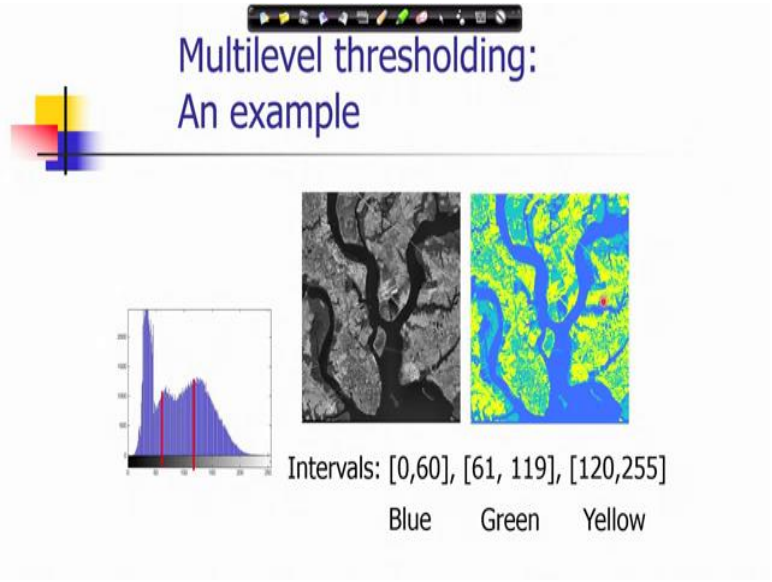
We will move to the next part of this particular topic. We are still over viewing some of the Fundamentals of Image Processing.

(Refer Slide Time: 00:25)



And we will start with the concept which is called Segmentation. Now, a segmentation is partitioning of image pixels into a meaningful non-overlapping sets. The binarization process what we discussed earlier, it is a special case of this segmentation. In that case, we have only two such sets that we discussed; one is of the background, another one is foreground. But it could be extended to groups of pixels, which are more than two levels.

(Refer Slide Time: 01:01)



For example we can consider a multilevel thresholding scheme. In the previous case, we have just only single thresholds. In a multilevel thresholds, we can have more than one threshold. Say consider this particular image and its histogram and let us consider we have selected these two thresholds. We can use a similar computation what we discussed earlier, particularly the Bayesian classification methods that can be extended further and we can determine this thresholds automatically also or manually by visualizing the modes, you can also select this thresholds.

In this particular case, let us consider there are three such intervals and for each interval, we assign a particular colour to those pixels. There are 3 colours you can look at this particular image and they are blue and green and yellow and each colour represents a segment of the image. As we can see from the intervals say there are 0 to 60 and 61 to 119 and 120 to 255.

$$Intervals: [0,60], [61,119], [120,255]$$

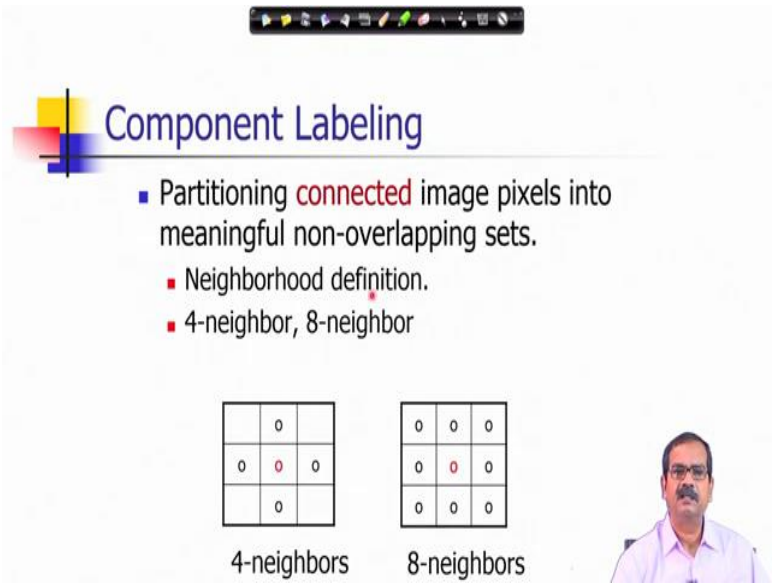
$$for \quad Blue, Green, Yellow$$

These are the definitions of those non overlapping intervals and this is how the pixels are grouped into this particular image into 3 different segments.

You can note that 0 to 60 corresponds to the darker values which is the water channel here. 61 to 119 is a middle values. Usually, these are the points which corresponds to this

interval and 120 to 255, they are the brighter values and they corresponds to the yellow portions of the image. So, this is the blue zone. This is the green zone and this is the yellow zone.

(Refer Slide Time: 02:57)



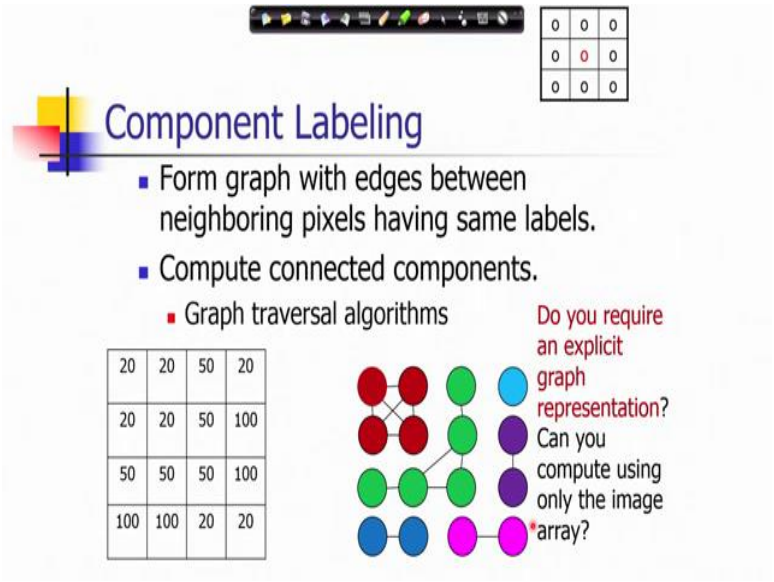
So, one of the task of segmentation even after grouping the pixels into different intervals that is still remaining and that is called component labelling. So, in this case, we need to also partition the image pixels in the plane itself. So, it is a connected image pixels into meaningful non overlapping sets that is what is our objective. So, there we can use various neighbourhood definition to define this connectedness.

For example, we can consider the 4-neighbourhood definition or 8-neighbourhood definition in this respect. Say 4-neighbours are defined in this fashion. Say given a pixel, these are the positions of the neighbouring pixels and out of them, these are the pixels which are called 4-neighbours. Suppose, this is a location x and y. This is a coordinate location. So, this could be x, this should be x plus 1 y. So, this is x right neighbour. Similarly, x minus 1 y, this is its left neighbour. x y plus 1, its top neighbour. x y minus 1, its 4-neighbour, its bottom neighbour.

So, these are the 4-neighbours of this pixel and this neighbourhood is called 4-neighbourhood whereas, now if I consider also the diagonal pixels, diagonal positions are also their neighbours. So, then this definition is an 8-neighbourhood definition. So, you

can have different such neighbourhood definitions and accordingly, this connected, connectivity will also be defined between 2 pixels.

(Refer Slide Time: 04:35)



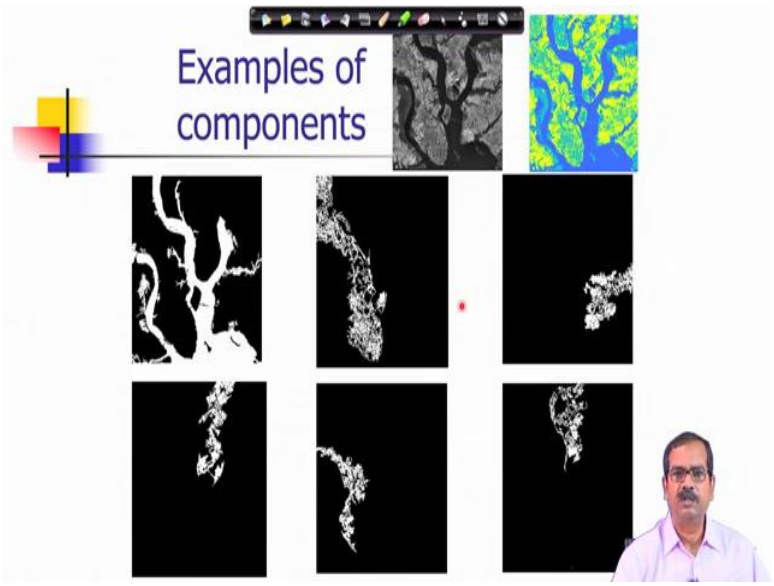
So, let us discuss a particular case when you want to get the components preserving the 8 connectivity of pixels. So, let us take this particular example. So, this is a small image and each pixel value is represented by these numbers. So, this is a 4 x 4 image for example. So, what we do, we can form a graph with edges between neighbouring pixels having same labels which say you consider every pixel is a node of this graph and you form the edge when their neighbour, the neighbours have the same pixel values.

So, for example, this is 20, 20, 20, 20. So, you can see that all these pixels, they form edges among between themselves and similarly, we can form edge between 50, 50 and this pixels in this form and also say for 100, 100 this is a edge. For another case this 20, 20 form an edge and 100, 100 another edge. So, it gives graphical representation of this particular image following the neighbourhood definitions.

So, now your task would be to find out the components of these graphs, you can use any graph traversal algorithms like breadth first search or depth first search and by which you can find out each component and declare them as a segment. So, I have displayed here by different colours, those segments. They are showing different components in this case. One of the interesting question, what could be asked here that whether you required explicit graph representation in this case.

Because we have already the image array and the neighbourhood definitions are merely précised, where they have a well organised structure. So, actually do not require any graph representation. You can compute everything on this array itself to get these components. So, you can compute using only one image array itself.

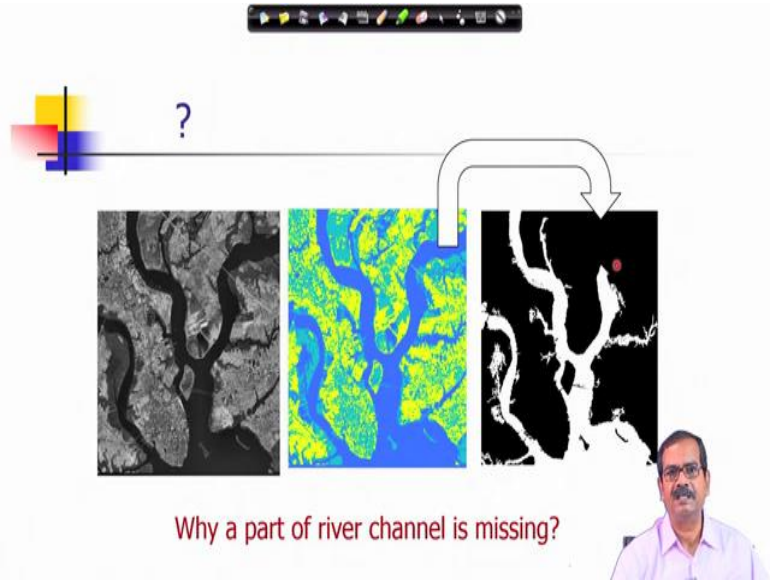
(Refer Slide Time: 06:59)



So, some of the examples of this processing let me show you. Say these are the two images which we earlier displayed. So, this is the original image and this is the segmented image into 3 segments, 3 classes of intervals. Now, we are forming the connected components from each segments. So, I will show you some of the prominent components in order of their number of pixels in those components. So, this is a largest component and you can see that in this component, we could recover we could get the river channel into one component itself.

Similarly, you can have various other components. So, this should be corresponding to this part of the zone. This is a green segment. So, connected green segments are shown here. This is mostly showing you connected regions in this part, yellow segment part. So, you get brighter pixels, yellow segment parts from here and in this way there are various other components could be also retrieved. I have shown only 6, but there are many other components which are possible in this image.

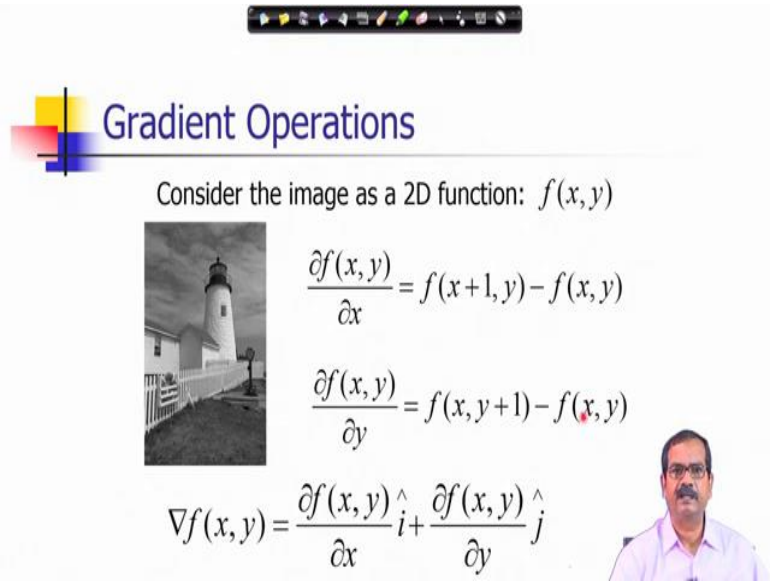
(Refer Slide Time: 08:15)



One of the interesting question that I can ask you from this results itself; you need to be carefully observing particular, you should observe this results. See you can find out that though in the original image, there is a river channel, this is a river channel and in the largest segment also you can get the corresponding pixels to river channel, but part of the river channel is missing in this segment.

So, can you tell why this part is missing? So, this is a question you need to find out by yourselves and you check why this information is missing in this part.

(Refer Slide Time: 09:05)



So, let us now discuss another operations for this with the images and we called this operations are computation of gradient values in the images. So, a gradient operation could be is defined in this way. So, image is a 2-dimensional function. So, in a 2-dimensional functions a gradient vector is defined by the corresponding partial derivatives along two principle directions. So, this is a partial derivative with x

$$\frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

and partial derivative of function with y

$$\frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

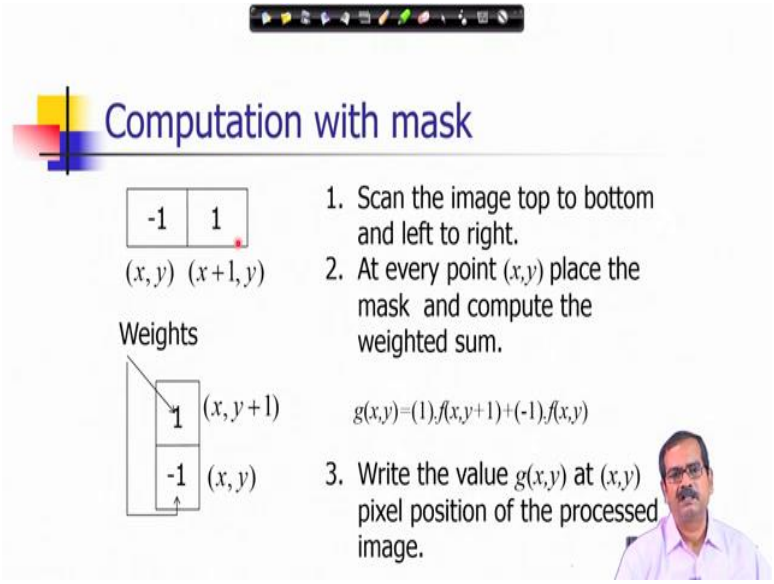
and they form a vector

$$\frac{\Delta f(x,y)}{} = \frac{\partial f(x,y)}{\partial x}\hat{\imath} + \frac{\partial f(x,y)}{\partial x}\hat{\jmath}$$

and that is a gradient vector.

So, computation of this gradient is quite simple and direct. We can use finite difference method by which you can compute gradient along the x direction which is simply the difference between the right neighbour and the pixel itself and similarly, we can compute the gradient along y direction. It is a difference between the top neighbour and the pixel itself. So, functional values at those positions is a difference. So, with these simple computations, you can compute these gradients and you can get the gradient vector.

(Refer Slide Time: 10:21)



These computations, so let me discuss in a more general framework. There is a motivation behind this that would be understood later on. Let us consider that these difference operations could be performed by a computation with mask. So, let us define a mask. In this case its a mask 1 x 2 elements. Mask is a kind of array. In this case, it happens to be 1-dimensional array. So, the dimension is that there are 2 rows and number of column is 1 in this case.

And these values in the mask, they represent the weights. So, what you need to do for computing the gradient all on the vertical direction, you place this mask at every pixel positions. So, this particular one of the points in the mask is corresponds to the central pixel positions and that is shown here by this particular coordinate (x, y). So, you are placing this mask in the (x, y) positions and then, what you are doing you are performing the computation. So, you are performing the weighted sum of the pixel values of the corresponding with its neighbours, where this mask is also present.

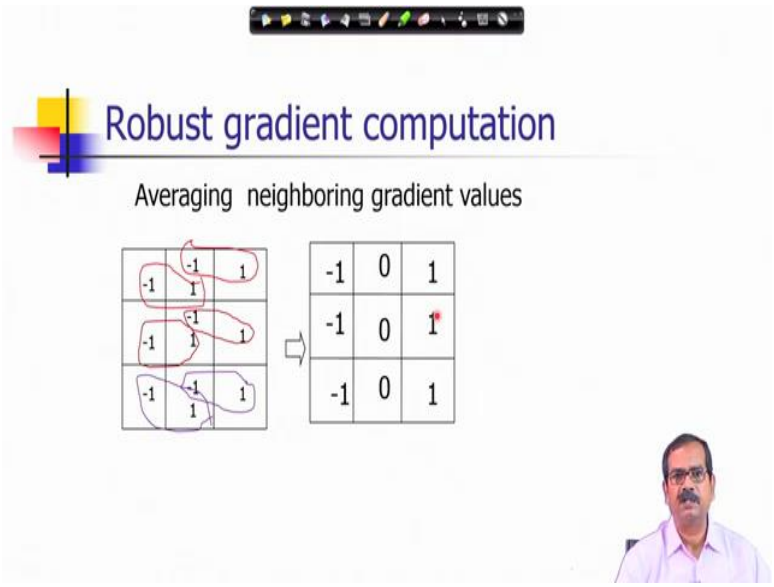
So, once you place a mask here (x, y). So, this should be my multiplied by - 1. Then, the functional value at (x, y +1) should be multiplied by 1. So, effectively if you take the sum that would be the difference between (x, y+1) and (x,y). So, the algorithm for these mask would be that you scan the image top to bottom and left to right, then at every point (x, y) place the mask and compute this weighted sum which means that as I mentioned that you multiply the weight with a corresponding functional values at those locations and then,

you take the addition. Take the sum of them and that would give you the weighted sum and then, you replace the central value by this weighted sum.

$$g(x, y) = 1.f(x, y + 1) + (-1)f(x, y)$$

So, in this way you can compute the finite differences along the vertical directions. Same computations can be carried out with this mask also which computes that difference along the horizontal directions.

(Refer Slide Time: 12:37)



So, to generalize this computation for making a robust gradient computation, what we can do instead of computing the difference only with the right neighbour or top neighbour, we can consider a neighbourhood region and find out this statistics of this finite differences in that region and take the mean of those know mean of that distribution.

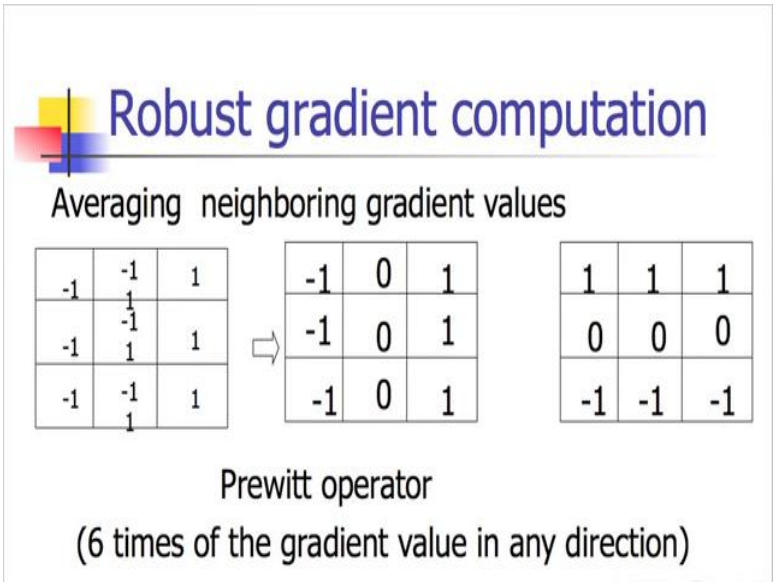
So, in this case particularly we are representing the mask value. So, we are taking a 3 x 3 mask and this is a central positions, these (x, y) positions. So, with respect to these position, so we can compute finite differences along using these two pixels also. So, , we are computing these finite differences 6 times around its neighbourhood and each one would not be the same; they are different values.

But it is expected that if there is a gradient directions statistically I mean they if you take the average, then the noise or the error would be reduced, if you take the average of all these values. So, which is equivalent this computations since these are all linear

combinations, this computations can be carried out by single mask computation itself, when the mask is designed in this fashion.

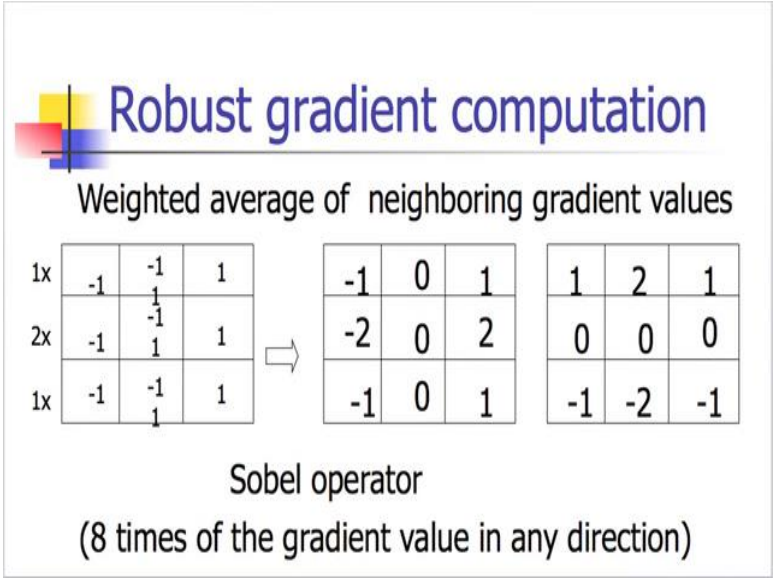
So, we can note that what we are doing at every cell of the mask, we are simply adding the weights. So, 1 - 1 becomes 0 here. This one is 1. So, in this way this is a distribution of weights in the mask and we can carry out the same computations with this mask and compute the corresponding. So, horizontal gradients in this particular case, in the gradient in the horizontal direction.

(Refer Slide Time: 14:37)



Similarly, gradient in the vertical direction also could be computed by this mask. Incidentally this masks two masks are called Prewitt operator in image processing and as you understand that 6 times you have computed this gradients in a particular direction.

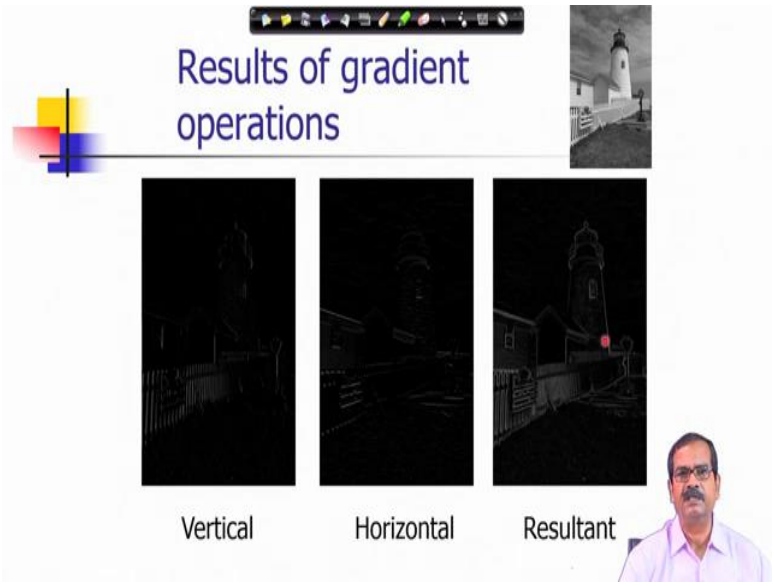
(Refer Slide Time: 14:59)



There is another example of robust gradient computation. In this case, what we are doing we are giving more weights to the central gradients than the off centric gradients. So, for example, in the horizontal directions when we are computing this gradient, we are giving weights of 2 here.

So, weights are doubled compared to the off centric rows and once again as I mentioned that every value of the cell can be computed by the sum of these corresponding values and if I multiply them with the weights. So, the distribution of the weights in the mask will look like this. This is for the computation on the horizontal directions and similarly, for the vertical directions, the distribution will be here in this case and so, this is a Sobel operator and this gradient has been computed 8 times in this case.

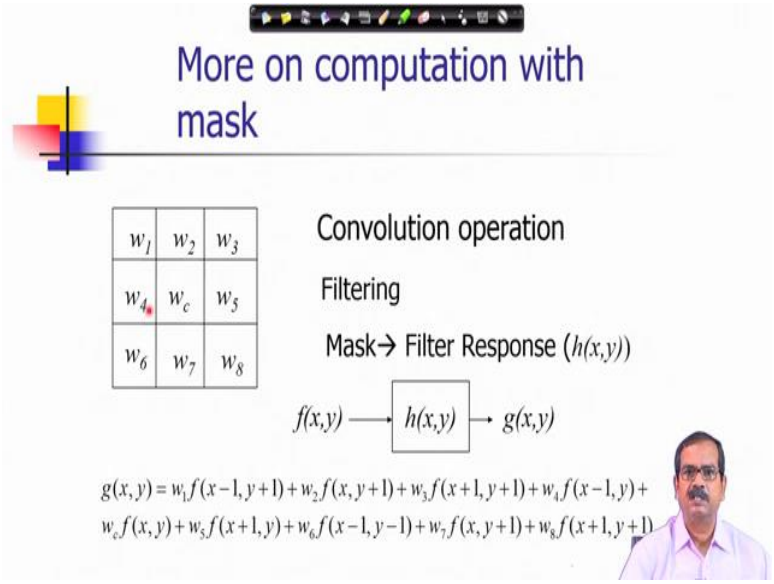
(Refer Slide Time: 15:57)



So, the results of these gradient operations, let me show you here. Consider the image on right top is your original image and if I compute the vertical gradients, I am showing those pixels by vertical gradients are very prominent. You can see those vertical bars of the fences; they become prominent in this particular image. For horizontal gradient again the horizontal directional fences which are parallel to the ground in those directions, those fences are becoming more prominent.

But if I consider the resultant of this two vectors by taking the magnitude of the vector itself, we will get all the edge pixels in every direction and the corresponding edge points or boundary points of different object services, they become prominent in this particular image. This is how you compute that gradients.

(Refer Slide Time: 16:53)



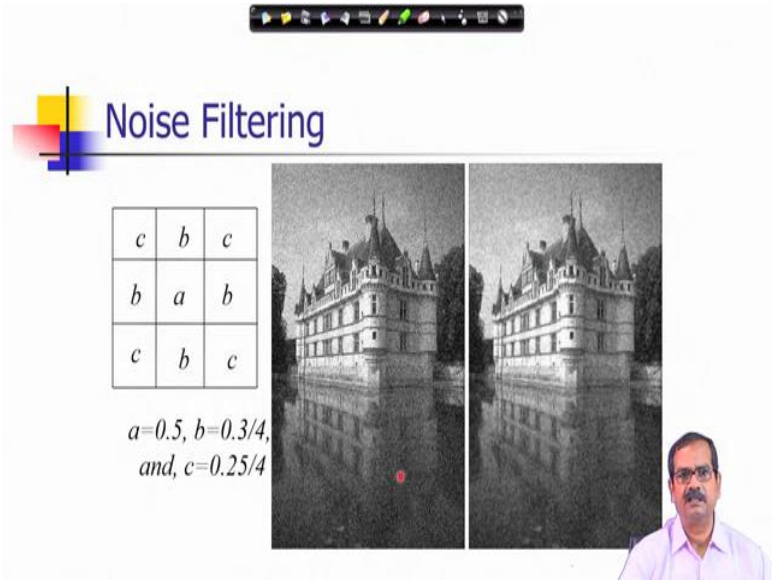
Let us generalize these computations with any arbitrary mask; that means, we consider any arbitrary weight distribution and this is an example of a 3 x 3 mask, but it could be of any dimension itself. So, if I consider weighted sum of the pixel values around the neighbourhood of the central pixel and we can represent this computations in the form

$$g(x,y) = w_1 f(x-1,y+1) + w_2 f(x,y+1) + w_3 f(x+1,y+1) + w_4 f(x-1,y) + w_c f(x,y) + w_5 f(x+1,y) + w_6 f(x-1,y-1) + w_7 f(x,y+1) + w_8 f(x+1,y+1) +$$

So, we are assuming that we are considering the functional values at the corresponding locations and then, multiplying its weights and we are taking the sum of all these values and that gives you the corresponding weighted sum and which would replace the functional values in the processed image. So, this computation is nothing but a convolution operation, where at every pixel you are placing this mask and you are computing this weights and then, replacing that functional value with this processed value.

So, it is the convolution operation. It is an operation meant for a computing outputs given an input of a linear shift invariant system in this case and this is an impulse response. So, these weights they define the discrete impulse response in this case, which is also called filtering because there is a frequency domain, transform domain interpretations which i will discuss in the next lecture. So, this mask is sometimes called also filters. So, this is the filter response h(x, y).

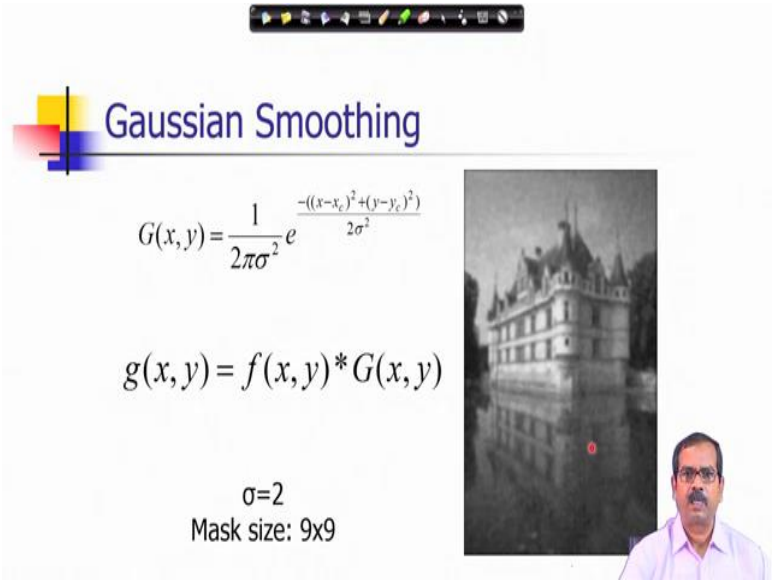
(Refer Slide Time: 18:39)



One example of this kind of filtering is noise filtering, where the mask values are shown here. You can see that every value is positive and its weighted combination of the neighbouring values. Finally, sum of all these weights is equal to 1. So, this gives you a kind of low pass filtering action and if I apply this operation, these convolution operations on this image with the same mask computations what we discussed that is weighted sum of the weighted sum of pixel values around its neighbourhoods.

And from there, replace the original pixel value by the processed pixel value, we will get an image something like this, where you can see that this noisy structures look little reduced in this case..

(Refer Slide Time: 19:33)



A special case of this kind of filtering operation often used in the image processing tasks is called Gaussian smoothing.

$$G(x,y) = \frac{1}{2\pi\sigma^2}e^{-\frac{((x-x_c)^2+(y-y_c)^2)}{2\sigma^2}}$$

When the filter or the mask is computed, the weights of the mask is computed following a Gaussian distribution. So, you consider the origin of this distribution at the central pixels and there is a width of the Gaussian distribution or standard deviation of the Gaussian distribution. Accordingly, the mask size also has to be determined to capture the Gaussian most of the functional values, significant functional values around its mask.

So, * is a symbol which is usually used for convolution operation. We denote this computation of with the mask this is what is a convolution operation.

$$g(x,y) = f(x,y) * G(x,y)\ with\ \sigma = 2, mask\ size: 9X9$$

So, in this particular case, if the mask size is 9 x 9 and your σ is 2; then, using this know Gaussian mask by using this convolution operation, you can get a result like this.

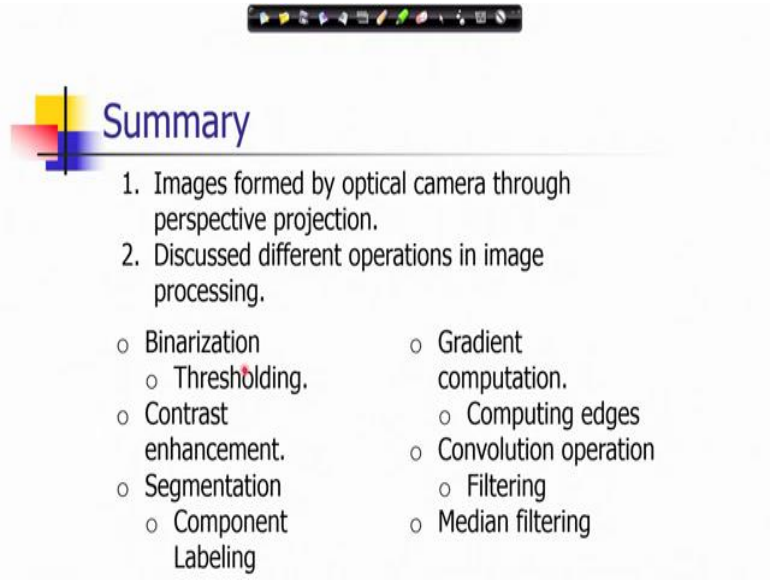
(Refer Slide Time: 20:43)



Median filtering is another kind of computation which is not which does not use convolution rather it is a very simple to describe. What it does, it computes the median value among the neighbours among the neighbouring pixels and replaces original value by this median value.

It is a kind of non-linear filtering operations, but this is a very effective method to reduce a noise like spot noise kind or salt and paper noise kind of noise which are randomly the random dots which can be scattered over the images and those could be minimized, those could be reduced using mitigated using this kind of filtering.

(Refer Slide Time: 21:35)



So, here we are at the end of our lecture on this fundamentals on image processing and so, as a summary what we discussed in this particular lecture that we can summarize it. That images they are formed by optical camera and we see that the mapping of 2-dimensional, 3-dimensional points, 2-dimensional points that takes place through a kind of projection which is known as perspective projection.

So, we discussed rule of projection. That in this case, a point from a scene point a 3-dimensional point, you have to draw a straight line which passes through a particular point which is known as centre of projections and which hits the image plane that is point of intersection defines the corresponding image point that is what is perspective projection. Then, we have discussed different operations in image processing. Some of them are say binarization and in binarization particularly, we discussed the thresholding operation. Then, we discussed also contrast enhancement, we discussed segmentation and special task of segmentation say component labelling.

We also discussed gradient computation where we have discussed how edges could be computed and convolution operations. In the convolution operation, we also introduced the concept of filtering which is equivalently called filtering and it is a computation with a mask; a mask consist of weights in its cells and that weighted sum gives you corresponding distribution and then, we also discussed median filtering at the end. So, here let me stop.

Thank you very much for your listening.

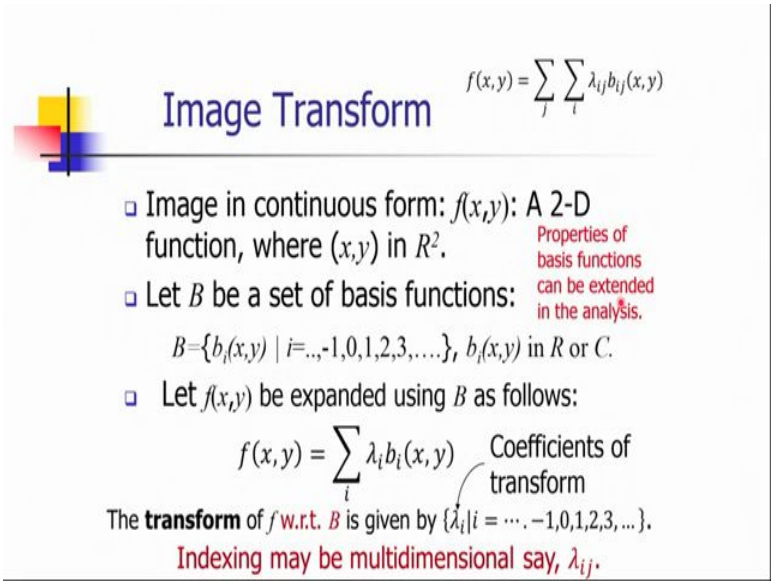
Keywords: Binarization, segmentation, gradients, convolution, filtering.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

Lecture - 03
Image Transform Part - I

In this lecture I will introduce Image Transforms.

(Refer Slide Time: 00:21)



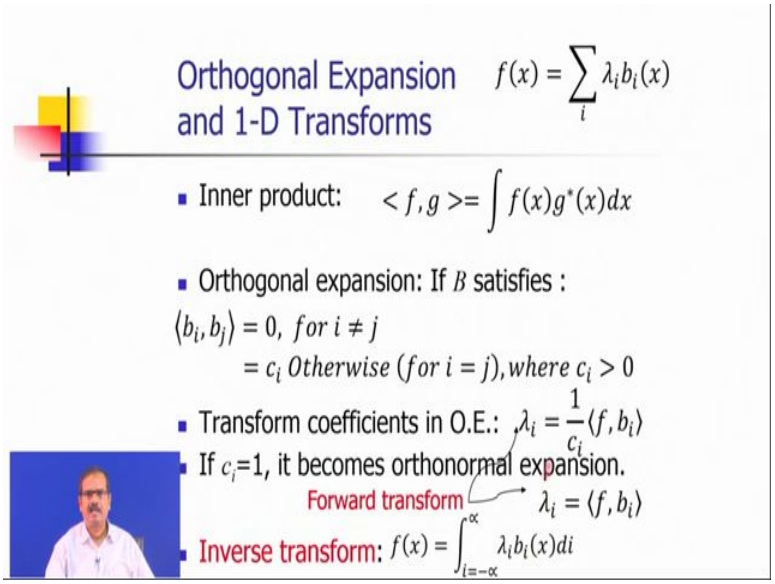
Let us consider image as a continuous function; it is a two-dimensional function and a point is in the two-dimensional real space and let us consider set of basis functions which are also a two-dimensional functions, we can represent it as a set where each function is given by say

$b_i(x,y)$. you should note that this could be the functional value, could be either in real or in the complex domain. We can represent, we can expand the two dimensional function f(x,y) using B as a linear combination of this basis functions as it is given here in this form that $\lambda_i \times b_i(x, y)$ and where i is the indexes of the basis functions as given in the set.

So, the transform of f with respect to B is given by the set of set of this coefficients which are λ_i these are called coefficients of transform and we can represent the function as the linear combination of this basis functions where the coefficients of the linear combinations are listed here. So, you can see that this is an alternative description of the image instead of representing the image by the functional form of f(x, y); I can simply represent it as a list of coefficients or even these coefficients can be a function of indexes.

So, indexing maybe multidimensional, for example for a two-dimensional function indexing we can use two such indexes to denote a coefficient and in that case we can say that this function can be expanded in the form of a linear combination of two dimensional functions and we can have this double summations in this representation. So, one of the advantage of image transform is that this properties of basis functions this can be extended in the analysis.

(Refer Slide Time: 02:37)



Let us consider a particular type of property which is very useful when this basis functions they have this property. This property is called orthogonality property and if I expand the function in terms of this orthogonal basis functions then we call that expansion as orthogonal expansion. We will consider our discussion following up the discussion on

this image transform, we will restrict our discussion to one-dimension first and later on we will see that now these properties can be easily extended to two dimension. So, we will understand the one-dimensional transform initially.

So, one of the thing that we would like to define here this operation inner product. So, inner product is a binary operation where two operands are two functions you can see that this function f(x) and the another function g(x). So, it is the product of this two functions and integral of this product values at every point in the space x. Now of occurs in this product there is a there is there is something we should note that it is not a simple product, it is a product of function f(x) with the complex conjugate of g(x) and that is we are considering here. If both f(x) and g(x) functional values are in the real domain then complex conjugate itself will be the same functional value, so then we can write it as f(x) g(x) dx.

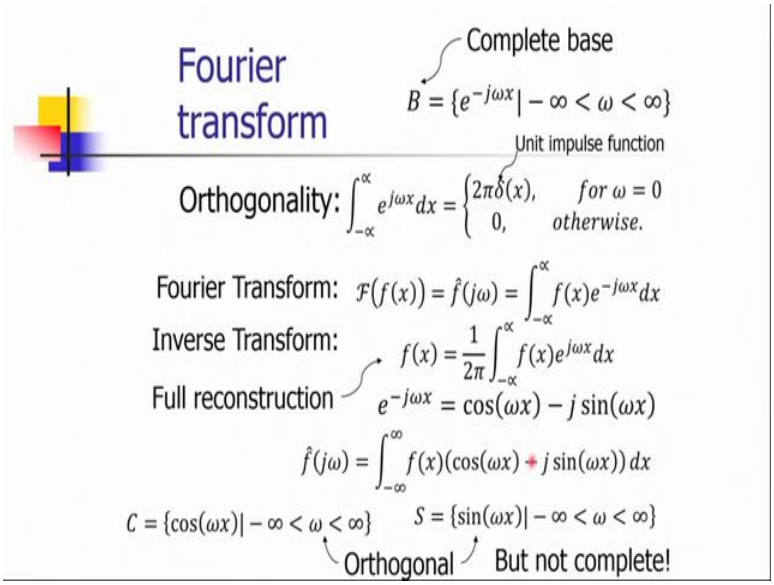
So, orthogonal expansion it is possible when the basis functions this satisfies certain property, that means the set of basis functions it satisfies this particular property of a orthogonality. What we can see that inner product of any two different basis functions that should be equal to 0, whereas inner product of the same basis function will have a non zero value and which is a positive value. If this is true for any pair of basis functions in the set B, then we say that this basis functions they are all orthogonal in that set.

So, transform coefficients in orthogonal expansion that could be easily computed by exploiting this property, that is one of the usefulness of this particular property and you can say that simply if I take the inner product of the function with a basis function, then we can and also divide it by c_i then we can get the corresponding λ_i's. And if c_i is equal to 1 then it becomes orthonormal expansion then we can simply write λ equal to inner product of function and b_i. So, this is what of since we are expressing the functions in terms of only transform coefficients, so we call this operation is a Forward transform operation.

So now, function instead of being represented by f(x), now the functions function is represented by these λ's and the reverse transform or inverse transform would be to compute the function from this coefficients back and because of the orthogonal property and also

orthonormal property, we can simply write it in this form. That means, simply it is a; it is nothing but the linear combination of those basis functions and since it is a continuous domain. So, we are taking the integrations over all the index values otherwise in a discrete domain we can write it in this form.

(Refer Slide Time: 06:29)



So, one of the special case of this orthogonal expansion is Fourier transform and in this case you can see that set of basis functions is given by in the following form:

$B = e^{j\omega x} | -\infty < \omega < \infty$ (eq. 1)

it is a complex sinusoid which is the member of the set, it is given in the above form and the completeness (Refer Time: 06:53) of this basis function is : any orthogonal set which is a subset of any orthogonal basis set will also remain orthogonal, but using the linear combination of that subset, you will not get the complete reconstruction of the function.

The basis set which keeps the complete reconstruction that is called the complete base. So, in the Fourier transform, in fact, the set what is defined here it is a complete base because, it can give me back this function as a linear combination of this function. You can see that

actually this (eq. 1) is a infinite set, though individually every sinusoid can be distinguished here.

So, the orthogonality property is reflected by this particular relationships where you can see the that □(x) is the unit impulse function whose area is equal to 1 centering at ω equal to 0 and otherwise □(x) value would be 0 in everywhere. So, it is an unit impulse function and this particular property gives you the orthogonal property of the Fourier transform this base.

So, Fourier transform can be defined in the following form which is the forward transform .

$$F(f(x)) = \hat{f}(j\omega) = \int_{-\infty}^{\infty} f(x)\, e^{-j\omega x}$$

as you see in the above that it is an inner product of f(x) and also the complete base. So, the base is $e^{j\omega x}$,. So, you take the complex conjugate of the base which is $e^{-j\omega x}$. And if I take the inverse transform then once again this is considered as the inverse transform;

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(j\omega) e^{j\omega x}\, dx$$

So, $\hat{f}(j\omega)$ is the corresponding coefficients and the linear combination of this basis function will give you the corresponding inverse transform.

So, it gives you the full reconstruction because it is a complete base as I mentioned. One of the interesting fact that can be noticed in the following complex sinusoid

$$e^{-j\omega x} = cos(\omega x) - jsin(\omega x)$$

it can be recomposed into two real and imaginary parts, real part consists of $cos(\omega x)$ and imaginary part consists of say $-sin(\omega x)$ in this case. So, this forward transform can be expressed using the following expression itself.

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(j\omega)(cos(\omega x) - jsin(\omega x))dx$$

So, it has one transform component which consists of real part, another transform component which consists of imaginary part and in the real part we are using the basis functions as $cos(\omega x)$ where as in the imaginary part we will be using the basis functions at $sin(\omega x)$ or $-sin(\omega x)$ whatever be your interpretation.

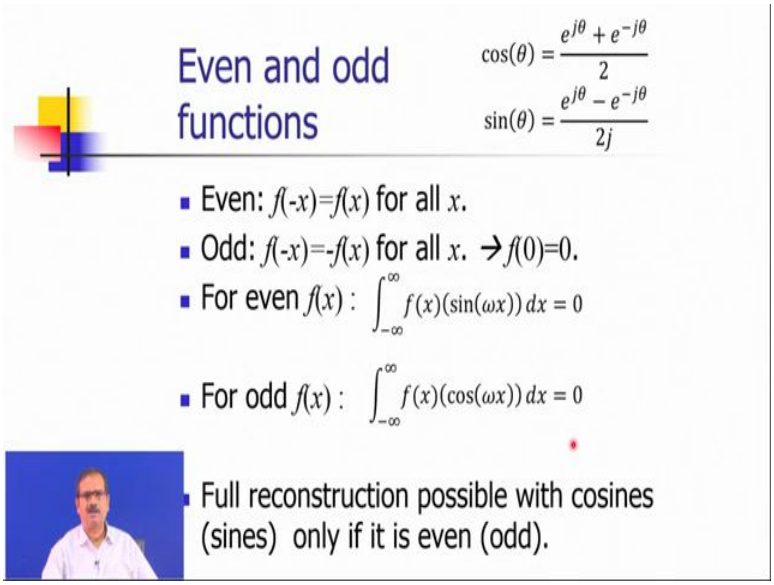
So, we can consider $cos(\omega x)$ as the set of basis functions and this is also orthogonal,

$$C = cos(\omega x) | -\infty < \omega < \infty$$

$$S = sin(\omega x) | -\infty < \omega < \infty$$

The above trigonometric functions are also orthogonal we know and $sin(\omega x)$ is also orthogonal. But the thing is that as I mentioned that they will not form the complete base. So, we can use only $cos(\omega x)$. If I use $f(j\omega)$ $cos(\omega x)$ the real part of $cos(\omega x)$ then that will not give you back the full function. Similarly if I use the corresponding imaginary part of the transform and use the $sin(\omega x)$ that will not also give me back the full function.

(Refer Slide Time: 10:55)



So, it is not a complete base but there are certain functions where actually if you use only cosine functions or sine functions, you can reconstruct it fully, so these functions are called even and odd functions.

There is a property like for an even functions, it should satisfy the following property:

$f(-x) = f(x)$ for all x

i.e, it should be symmetric around the origin or around at $x = 0$ at both ends say $f(-x)$ should be equal to $f(x)$ for all x

$f(-x) = -f(x)$ for all x

Whereas for odd it should be antisymmetric. i.e, $f(-x)$ equal to $-f(x)$ for all x and

$f(0) = 0$

naturally at the $x = 0$, the value has to be equal to 0 for this definition.

So, a function could be even, it could be odd or it could be neither, when they are belong when they have this property, then you can expand them using only cosine or only sine functions. So, let us see so even $f(x)$ we can have this is a property which is satisfied because, in that case if I take the integrations while taking the product with $sin(\omega x)$ that would be 0. I.e,

$$\int_{-\infty}^{\infty} \hat{f}(j\omega)(sin(\omega x))dx = 0$$

So, all sinusoidal terms would be 0; so that is why only cosine terms will remain and your transform coefficients can be sufficient to prescrib by only cosine transformations.

$$\int_{-\infty}^{\infty} \hat{f}(j\omega)(cos(\omega x))dx = 0$$

Similarly for odd $f(x)$ the above property is true and that is why using only sine sinusoidal basis you can reconstruct it.

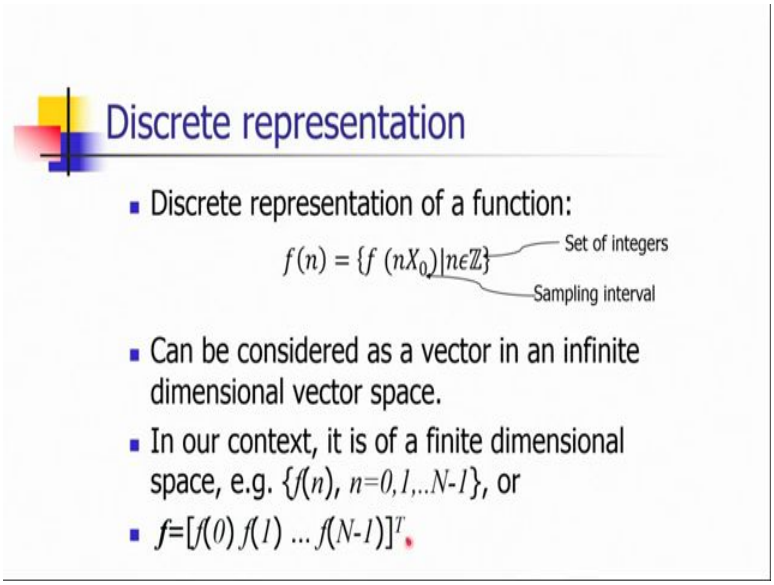
Now this could be easily derived if I consider this relationships of $cos(\theta)$ and $sin(\theta)$ in terms of the complex exponential quantities as below.

$$cos(\theta) = \frac{e^{j\theta}+e^{-j\theta}}{2}$$

$$sin(\theta) = \frac{e^{j\theta}-e^{-j\theta}}{2j}$$

So, full reconstruction is possible with cosines when the function is even and with sines when the function is odd.

(Refer Slide Time: 13:03)



Now, let us consider the discrete representation. So, a discrete representation of a function can be made in the following form :

$f(n) = \{f(nX_0) | n \in Z\}$

that the function needs to be sampled at periodic interval and it will provide a sequence of functional values where each sequence position is an integer set and sampling interval which is associated with this particular definition.

So, it can be also considered as a vector in an infinite dimensional vector space, but in our consideration since we will be always using images of finite dimension or the signals of finite dimension, you are representing them in the computation in the discrete domain.

So, there we will be having only a finite dimensional vector.

$f(n),\ n = 0, 1, ..., N-1$

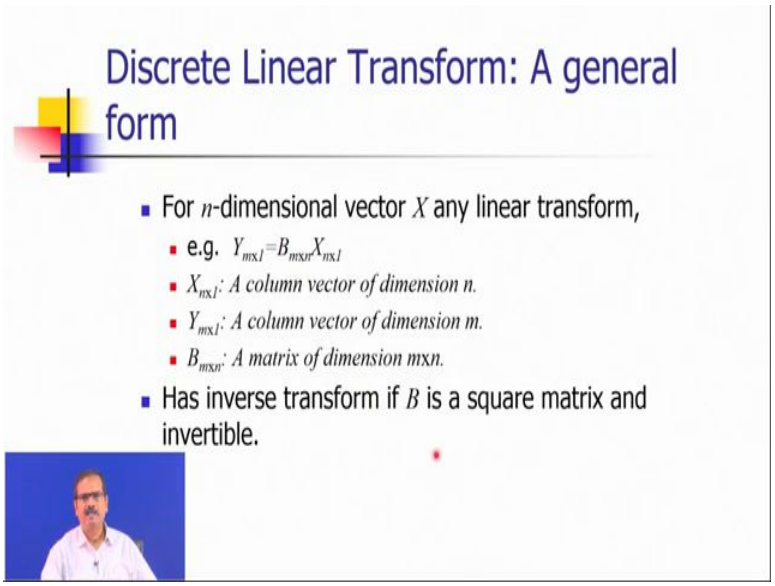
For example we can represent a function from within certain interval from say 0 to $N-1$ *as shown above*.

You should note here the sampling interval is implicitly represented in this form. So, even without sampling interval we have a representation of the function and when you are trying to interpret the function with the physical terms, physical quantities in the functional space then only the sampling interval has to be used.

So, let us consider that we are representing a function in this case with a finite dimensional vector and say it is an n dimensional vector in this case; it is a column vector representation. So, that is why the transpose operation has been used as shown below.

$f = [f(0)f(1)..f(N-1)]^T$

(Refer Slide Time: 15:05)



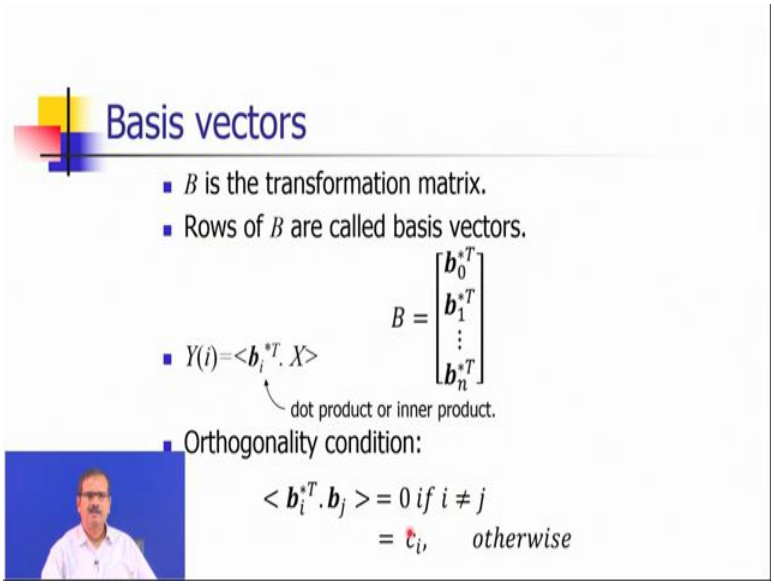
So, then how do you define a discrete linear transform? It is very simple because you know that whenever we perform any matrix multiplication in the following form:

$Y_{m\times 1} = B_{m\times n} X_{n\times 1}$

, say you have a column vector of n dimensional column vector and let there may be a matrix of $m \times n$ dimension and if I multiply them then you will get another vector of $m \times 1$ dimension, that means m dimensional vectors.

So, this is a transformation of this column vector into another column vector of a different dimension. we call it a linear transform or as it is discrete since we were using the discrete presentation, let us call it as discrete linear transform. And this transform has inverse when this matrix which is called say transformation matrix B is a square matrix and also invertible.

(Refer Slide Time: 16:11)



So, one of the interesting facts about this transformation matrix that we can note that we can consider rows of these transformation matrix B, they form the basis vectors, this is the analogy with respect to the basis functions. Because we will see instead of inner product between two functions we are having here, inner product of two vectors which is equivalent to the dot product of two vectors.

So, let us consider this say this is the representation of the transformation matrix

and these are the row vectors which is indicated by the transpose operations and you can say that this is a row vector you are considering, these are the complex conjugate operations by keeping it consistent with representation what we have for the inner product.

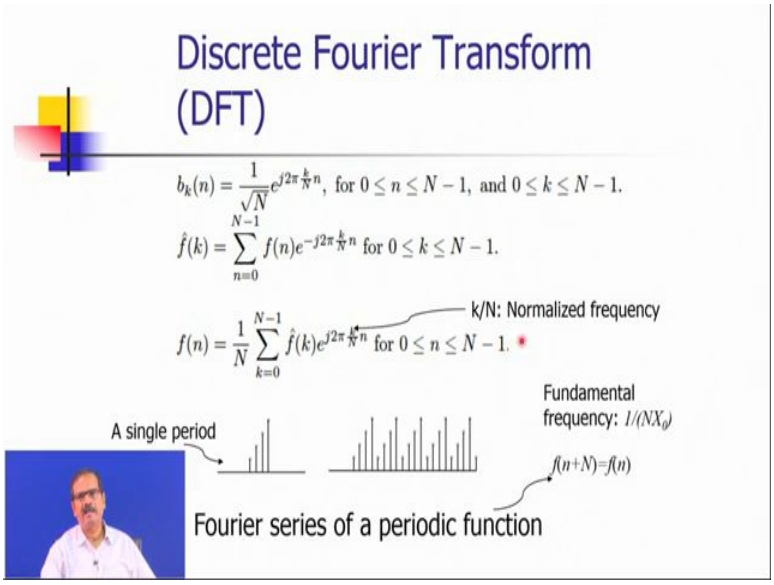
So, when we perform this dot product or inner product between two vectors, then you get the corresponding element. So, i^{th} basis vector provides you the ith element of the transformed vector which is Y here.

$< b_i^{*T} . b_j >= 0$ *if* $i \neq j$

$= c_{i,}$ otherwise

So, the orthogonality condition in this case is reflected in the above form such that if you take any pair of two basis vector, then their inner product should be equal to 0 if they are different otherwise they should have a non zero value c_i.

(Refer Slide Time: 17:43)



So, with this form we can consider that it will have the similar representations. You can consider that a function as a linear combination of all those basis vectors. So, if I look at the discrete Fourier transform expressions, the basis vectors for the discrete Fourier transforms are represented in this form. So, it is $\frac{1}{\sqrt{N}}\ e^{j2\pi\frac{k}{N}n}$ you should note here this is defined for n functional points.

$b_k(n) = \frac{1}{\sqrt{N}} e^{j2\pi\frac{k}{N}n}$ for $0 \le n \le N-1$, and $0 \le k \le N-1$

So, $b_k(n)$ is a small n^{th} element which means you can form a basis vector from by computing it $n = 0$ to $n = N-1$ at it at each integer value of n within these interval. So, that gives me a vector that is a kth vector and there are n such vectors where the k indexes k is indexes vary from 0 to $N-1$.

$\hat{f}(k) = \sum_{n=0}^{N-1} f(n) e^{j2\pi\frac{k}{N}n}$ for $0 \le k \le N-1$

So, forward transform or discrete Fourier transform of a discrete sequence $f(n)$ which is a finite sequence of length n can be expressed in the above form.

$f(n) = \frac{1}{N} \sum_{n=0}^{N-1} \hat{f}(k) e^{j2\pi\frac{k}{N}n}$ for $0 \le n \le N-1$

You can find out that this is nothing but the inner product of $f(n)$ *and* $e^{j2\pi\frac{k}{N}n}$ .only thing is that instead of putting $\frac{1}{\sqrt{N}}$ in this expression ,we have taken care of the square root operation during the inverse transform by multiplying $\frac{1}{N}$ as shown above .This is a simple operation .

So, we kept this division normalization operation. We removed that operation in forward transform and included it in during that reconstruction. So, actually the value what we will get that would be proportional and it does not matter at this stage. But when you reconstruct, it will again recover the same value because, you are taken care of that particular normalization during reconstruction operation.

So, you can see that it is a linear combination of the corresponding of basis vector in the reconstruction also and the coefficients which is given by $\hat{f}(k)$ are the coefficients from discrete Fourier transform. We can also observe from the above expression that discrete Fourier transform is nothing but Fourier series of a periodic function. So, let us consider a finite sequence. So, in this case for simplicity let me take only four functional values and so, which means my value of N is 4 here and this is a functions for which I will be doing discrete Fourier transform.

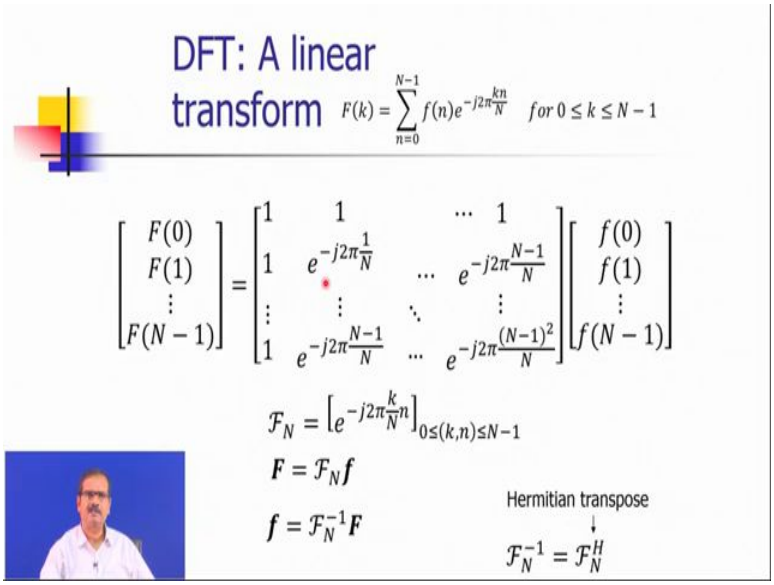
So, what I will consider because there is no definitions outside this interval, I can use any definition as per my convenience any other functional values and perform a transformation and after inverse transformation once again I will keep my observation window within the interval from 0 to 3 in this case. So, a periodic extension of this signal could be in this form that means, it is repeated so that you know in a periodic function this property needs to be a satisfied a periodic function with a period capital N should be $f(n+N) = f(n)$. So, that is satisfied as you can see here, here the value of N is equal to 4.

So, you will see after every fourth sample again it is repeating the same value. So, it becomes a periodic signal and as you know any periodic signal can be expressed as a linear combination of sinusoid functions and you can perform Fourier series ,that is ,what you are doing here in this case. After that, while in performing inverse Fourier transform , you are only performing for these four sample points. So, and just to note that how it is related with the actual physical sampling interval which is say X_0 here.

So, the fundamental frequency would be determined by the length of the period of this signal which is NX_0 So, fundamental frequency is $\frac{1}{NX_0}$.

And you can see that harmonics is represented by harmonic $\frac{k}{N}$ actual implicitly there is $\frac{k}{N} X_0$ that is the physical frequency. So, we call $k \times N$ as the normalized frequency in this representation.

(Refer Slide Time: 22:47)



So, discrete Fourier transform that can be also expressed in terms of a linear transforms which means we can express a discrete Fourier transform as a matrix multiplication of a column vector where the column vector is given by the functional values a finite dimensional vector as we have considered earlier and if I multiply these matrix then we will get the transform matrix and you can see that these elements they are obtained from this corresponding basis vectors definition.

So, we can represent this matrix in a shorter form where each k and N^{th} element is represented by the corresponding expression as shown below

$$\mathcal{F}_N = \left[e^{-j2\pi\frac{k}{N}n}\right]_{0 \le (k,n) \le N-1}$$

and if the value of k and n they range from 0 to $N-1$ which will give me an $m \times n$ matrix. So, in a forward transform what we are doing; We are simply multiplying this transformation matrix with a column vector f which is representing the particular column vector $[f(0)\, f(1)\, ..f(N)]^T$.

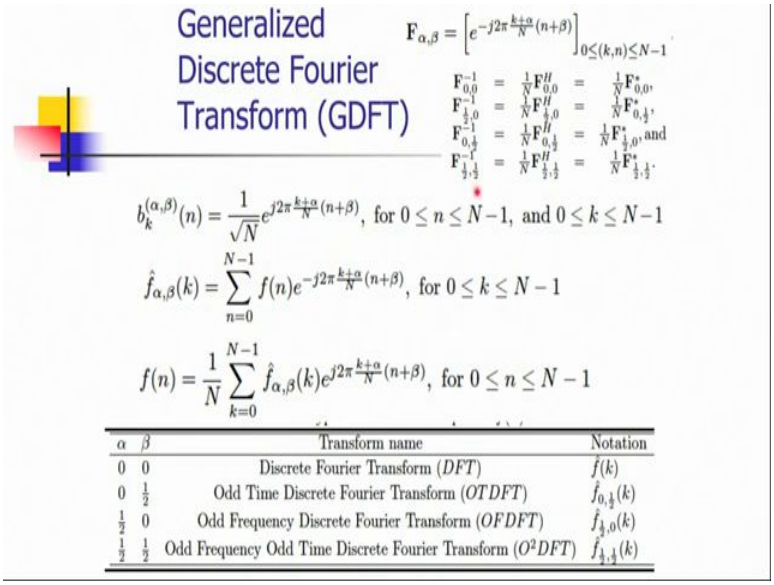
Then, we get the output transform matrix. So, the ($N \times N$) transform matrix transform column vector which is representing the column vector $[F(0)\, F(1)....F(N-1)]^T$ which is actually the coefficients of discrete Fourier transform. $F = \mathcal{F}_N f$.

and inverse transform will be naturally if I take the inverse of $F(N)$ and multiply with F then we will get back once again column vector. $f = (\mathcal{F}_N)^{-1} F$

Incidentally because of the orthogonality property and also orthonormal property of these functions. So, you can show that the inverse of the discrete Fourier transform matrix is nothing but its Hermitian transpose of the corresponding matrix. $(\mathcal{F}_N)^{-1} = (\mathcal{F}_N)^H$

Hermitian transpose is the transpose of this matrix and also you should have to perform complex conjugate operation that would give you the Hermitian matrix.

(Refer Slide Time: 24:53)



Now, discrete there could be other kinds of know say it off orthogonal basis vectors and some of them could be derived or extended from discrete Fourier transform representation. So, we call it generalized discrete Fourier transform.

If you can observe that in the discrete Fourier transform ,basis vectors are generated by sampling the complex sinusoid within an interval between 0 to $N-1$ and then we have sampled at regular interval .Now if I make a phase shift there in that interval. So, there itself we can have a variation instead of we can give a phase shift of β and also while defining basis vector we have considered harmonics and we have generated harmonics at regular interval.

$b_k^{\alpha,\beta}(n) = \frac{1}{\sqrt{N}}e^{j2\pi\frac{k+\alpha}{N}(n+\beta)}$ for $0 \le n \le N-1$, and $0 \le k \le N-1$

So, if I give also a shift in the frequency space ,then also you can have a different basis vector. Now the above representation will generate also n basis vectors of N dimensions and that would be once again an orthogonal basis vector which is a square matrix which could be invertible.

So, it could be easily used for once again for making a transform. So, this is the generalized discrete Fourier transform. We can use this basis vectors and we can get this expressions for discrete Fourier transforms. Similarly we can get back the function by applying the inverse Fourier transform it is the same similar form what we did for the case of discrete Fourier transform and the corresponding transformation matrix can be expressed in this form, here the elements as you can see it retains the same similar expressions only there are parameters α and β which is giving you a different set of transformation matrix.

There are some popular transformation matrix as you can see a special value of α at zero and β that itself will give you the discrete Fourier transform what we have discussed earlier. If I took $\alpha = 0$ and set $\beta = \frac{1}{2}$, we call that transform as Odd Time Discrete Fourier Transform or OTDFT and you can represent the transform in this form; similarly $\alpha = \frac{1}{2}$ and $\beta = 0$ would be Odd Frequency Discrete Fourier Transform and if both are half; Odd Frequency Odd Time Discrete Fourier Transform.

There are different properties which I am not discussing here; just for your example we have given this particular thing and they have their inverse transform in this case also. Those are related and I have shown you in this particular grade.

So, there are different relationships of the inverse transform. I think let us stop here at this point and we can start from this point in the next lecture, where will see that though it is not possible in the continuous domain to have cosine transform and sine transform for every kind of function but in discrete domain for any finite dimensional sequence you can define cosine transforms and sine transforms.

So, for that we will be using this generalized discrete Fourier transform. So, thank you for listening this talk and we will move over to the next lecture.

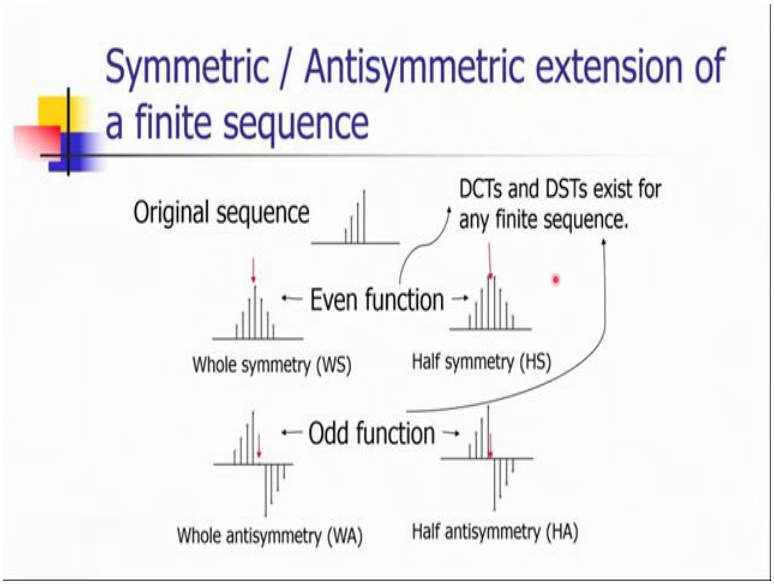
Thank you.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 04
Image Transform - Part – II

So, let us discuss about the second part of the Image Transform properties and in the previous lecture I discussed how discrete Fourier transform can be generalized. And then we will see using those generalized discrete Fourier transform, we can derive also discrete cosine and discrete sine transforms for discrete sequences and they form a complete base; so you can get a completely construction of those sequences.

(Refer Slide Time: 00:54)



So, to understand that fact let us consider a concept of symmetric or anti-symmetric extension of a finite sequence. As I mentioned for the case of discrete Fourier transform also that given a finite sequence; you can define this sequence in the zone where in the zone of undefined region as per your convenience, make that sequence having certain properties which will be useful and

which will be convenient for performing transformation on the sequences; on those extend sequences.

So, one such extension type of extension that is used in that is used for this purpose is symmetric and antisymmetric extension. So, let us understand what this is. you know symmetric extension and what is antisymmetric extension. Now you can see this is an example of a symmetric extension, as this was my original sequence and here the center of symmetry lies at the end sample of the original sequence itself.

And we call this kind of symmetric extension is whole symmetric extension or whole symmetry; a given a sample 4 actually can see that we require another additional 3 samples and by maintaining the symmetric symmetry of the sample values; we can create the whole symmetry. In the other kind of; no symmetry we can create that is called half symmetry, in this case central lies separated with an interval of half of the sampling period or sampling interval from the end sample; this is the point there is center of symmetry.

So, at this towards its left you have 4 samples towards its right also we have 4 samples; so this is called half symmetry. And for the antisymmetry we also similarly defined whole antisymmetry; in whole antisymmetry the antisymmetry, the center of antisymmetry lies at the at a sample value which is introduced; so this value should be 0.

And then next of the values should be you know determined by the corresponding sequence original sequence; they should be negative of the corresponding sample value and this is a whole antisymmetry. So, you can see the total length of in this case in this definition; it is 4 plus this is a value introduced 5 and then this is 4; so this is 9; this is a whole antisymmetry an example of whole antisymmetry.

Similarly, we can have half antisymmetry; so instead of introducing a 0 explicitly here; we can assume that center of antisymmetry lies in between half of the interval between these two samples. So, you do not require any additional introduction of 0; only the samples are inverted here and this is called half antisymmetry extension.

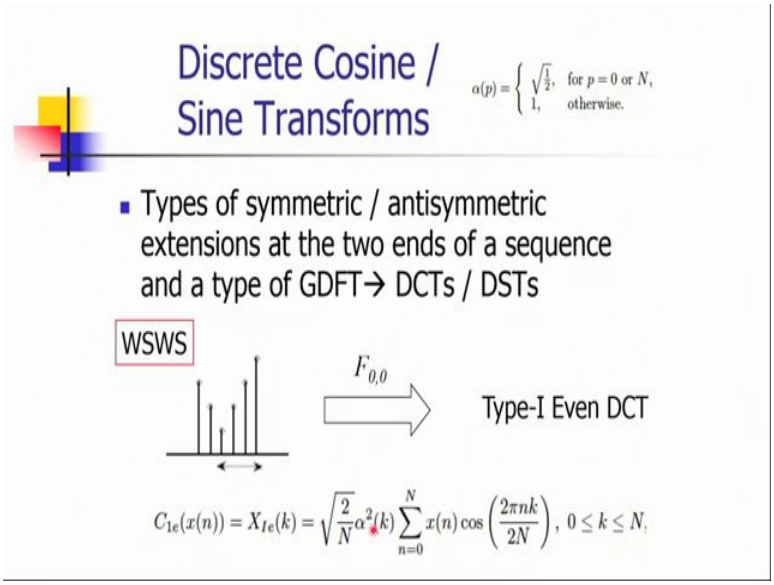
Now, the significance of the this symmetric extension is that, if I make the symmetric extension, then you can observe of this function becomes even function if we consider this is the sample value at the 0th index or x equal to 0 and it becomes also even function if your origin of x lies here. And similarly here it would be odd function once again if your origin lies here and otherwise in this case origin. So, now, as I was mentioning that given an original sequence actually, you have converted this sequence as even function and further if you consider periodic extension of this function; then you can apply discrete Fourier transform on this; some discrete Fourier transform.

And then what happens that since it is an even function that transformation will require only cosine parts; so, you can have discrete cosine transform using this kind of functions here. And again you can reconstruct back using that discrete cosine transform and keep your observation window only on this interval. So, you get the original sequence back.

So, in this way you get all the original sequence by just by using only cosine transforms. Similarly, for odd functions it is sufficient to use only sine functions because for odd functions we have seen that Fourier transform is equivalent to application of only sine functions from the basis function set, here also only the sine functions can be used.

So, that is what using this symmetric antisymmetric extension; we can have DCTs and DSTs or Discrete Cosine Transforms and Discrete Sine Transforms for any finite sequence. And that is the reason why do they exist and you find them in the text books and in many applications. So, for even function it is DCT and for odd function it is DST.

(Refer Slide Time: 06:26)



So, this is what I mean there could be various kinds of discrete cosine transforms because you have so many different varieties. you can have different types of symmetric or antisymmetric extensions at its two ends. And observe whether the sequence become even function or odd function and also you can use different kinds of discrete Fourier transforms from the definition of generalized discrete Fourier transforms.

So, you can have different DCTs and different DSTs; so take this example suppose we have a symmetric extension; whole symmetric extension both at the both ends, in that case we could observe that the functional values would be you know symmetric around this particular value. This is the end and this is the period that would be defined by this symmetric extension; if I extend it periodically with this one, we will get a periodic value.

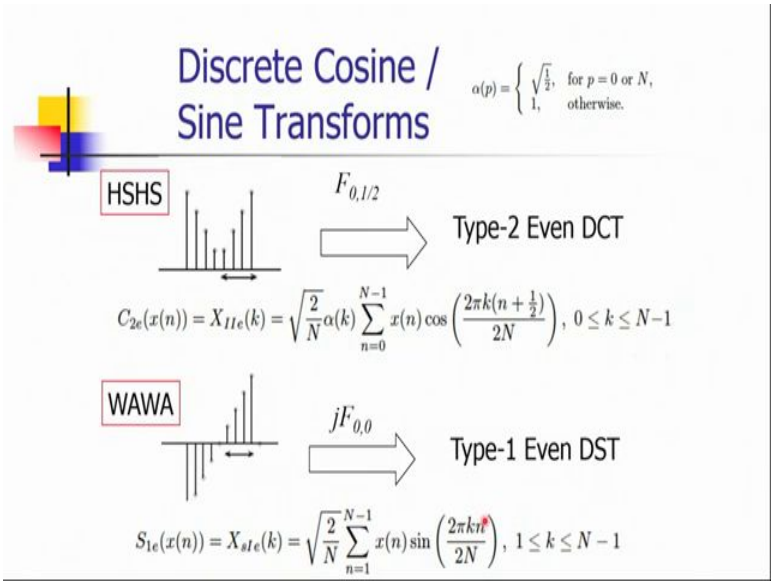
So, this is the original function and using the whole symmetric extension at this end and also at this end; you can identify this is the period, i.e, the minimum period that is formed by this particular extension. And if the value was 4; number of sample was 4 you can see that length of the period becomes 6. So, we will observe that how it affects the corresponding cosine transforms in this case.

It is possible to have a sine transforms and if we apply simply discrete Fourier transform on this extension will get a type I even DCT; whose expression is given in this form. You can say that only the cosine basis function is used; cosine functions is used from the based on the basis vectors. And you can see the period; period is 2N, but you observe the definition of N in this case the value ranges from 0 to capital N which means there are N+1 number of samples. So, if the value of N+1 is equal to 4. So, actual value of N is equal to 3 in this case. So, that is why the period is 6 right;

so this is how the type I even DCT is defined. There is a definition of $\alpha(p)$ also. This is for the normalization operations for making it orthogonal and orthonormal when you are performing the reconstruction; they should satisfy this property, so you will have this particular definition.

$$\alpha(p) = \begin{cases} \sqrt{1/2}, & p = 0 \text{ or } N \\ 0, & otherwise \end{cases}$$

(Refer Slide Time: 09:15)



So, similarly we can have other kinds of symmetric extensions HSHS. So, at both ends we have half symmetric extension and if this is your original samples that is of length 4; then half symmetric extension that both end will provide you a periodic signal of even function of length 8. And if I apply the alpha that general discrete Fourier transform, when α=0 and $\beta = \frac{1}{2}$ that is odd time discrete Fourier transform, then you will find it is a type II even DCT.

And the expression can be given in this form and here you can observe that given N samples you are generating a period of twice N. So, you have the following expression :

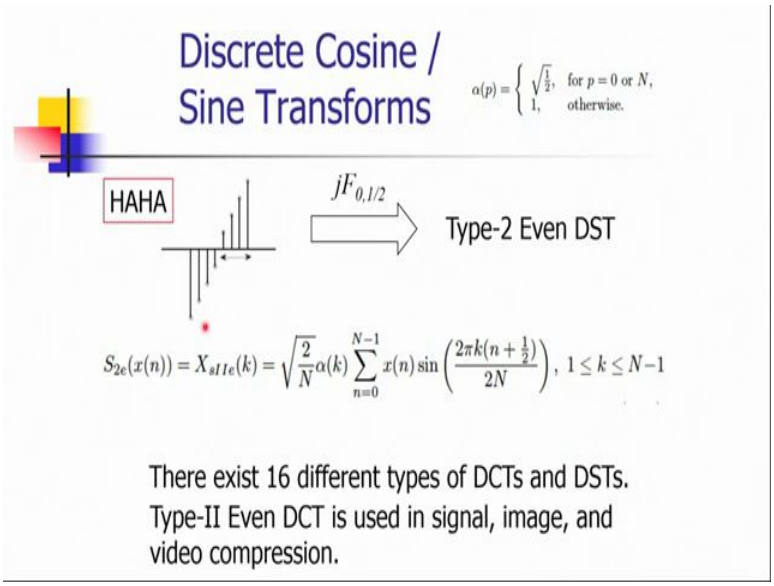
$$C_{2e}(x(n)) = X_{IIe}(k) = \sqrt{\frac{2}{N}}\alpha(k) \sum_{n=0}^{N-1} x(n)\, cos(\frac{2\pi k(n+\frac{1}{2})}{2N}),\ 0 \leq k \leq N-1$$

and this is a familiar DCT expression what you see in the text book and this is mostly used in image compressions and video compressions. And if I say this is type II even DCT is by default that would be considered as that discrete cosine transforms.

And for discrete sine transform similar we can have antisymmetric extensions; like whole antisymmetry extension and given a 4 samples; you generate a period of 9, it should be (2N + 1). So, we will define the definition of N instead of starting from 1; we should consider 0 we

should say N=1, N-1. So, actually your defining it with respect to N-1 samples and that would give you the twice N period and this is what your type I even DST.

(Refer Slide Time: 11:14)

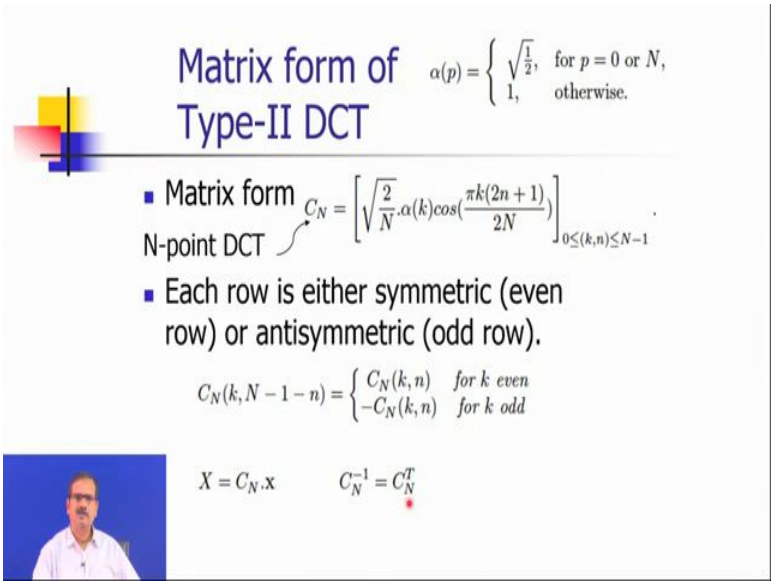


And for Type II even DST similarly, we have half antisymmetric extension and there also we are applying this corresponding discrete Fourier transform and this is a particular transformation matrix that you will be applying there. So, you will get a type II even DST and Following is expression for type II even DST .

$$S_{2e}(x(n)) = X_{sIIe}(k) = \sqrt{\frac{2}{N}}\alpha(k)\sum_{n=0}^{N-1} x(n)\ sin(\frac{2\pi k(n+\frac{1}{2})}{2N}),\ 1 \le k \le N-1$$

There would be many other DCTs and DSTs; in fact, since there are two ends and we can have two varieties of symmetry and two varieties of antisymmetry. So, in total there could be 16 different types of DCTs and DSTs and type II even DCTs is mostly used in signal image and video compression.

(Refer Slide Time: 12:10)



So, in a matrix form we can represent particularly type 2 DCT for an example I am showing all this transformation can be also expressed in the discrete linear transform what we discussed earlier. So, an element of the matrix that is $(k,n)^{th}$ element can be denoted in this form and this matrix is also referred to as N-point DCT matrix which is a type II DCT in this case.

And in this case, there are certain properties which are interesting and which are exploited in various developing different algorithms using DCT coefficients. One of this property is that : each row of this transformation matrix is either symmetric, we call the even row or antisymmetric. And following is a particular equations by which we are expressing this property

$$C_N(k, N-1-n) = C(k,n) \text{ for } k \text{ even}$$

$$= -C(k,n) \text{ for } k \text{ odd}$$

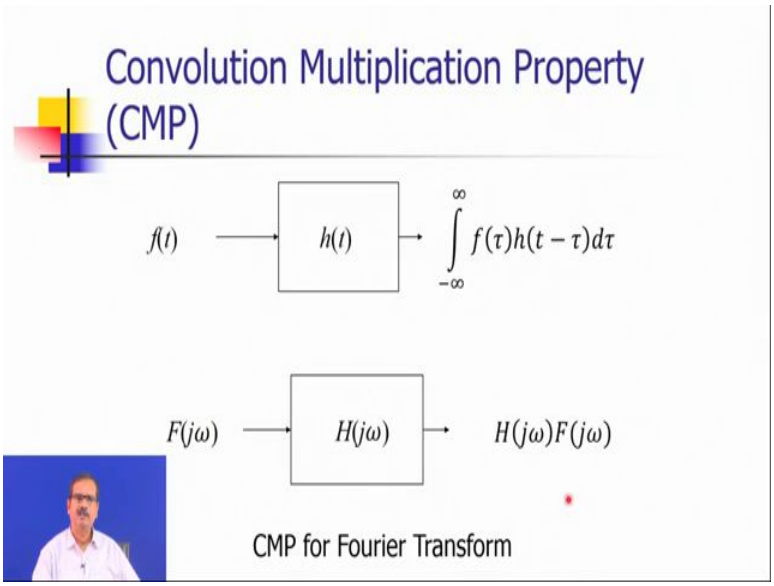
and the transformation can be expressed in terms of multiplication with the column vector $X = C_N x$

and you get the corresponding transformed DCT column vector and its inverse is also given in the following form

$$(C_N)^{-1} = (C_N)^T$$

in this case it is orthonormal expansion; so you can just have the transpose operation.

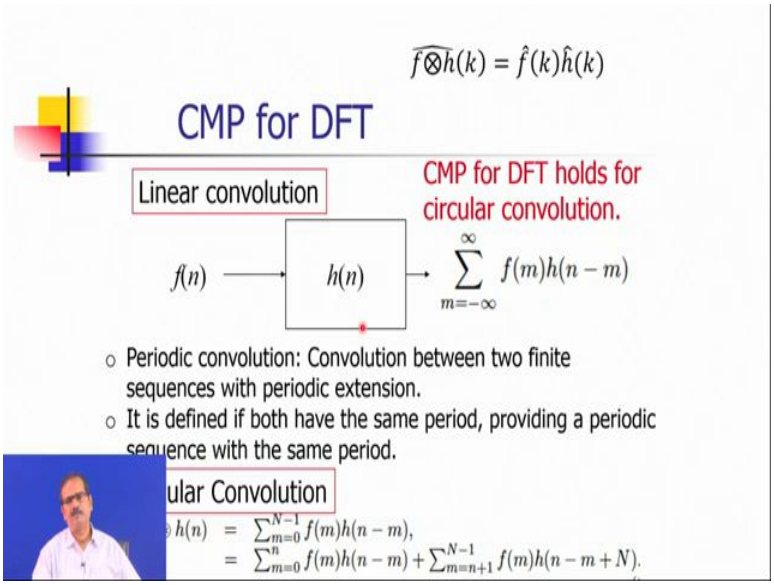
(Refer Slide Time: 13:24)



So, one of the application is that; it can simplify the convolution operations; so what you perform in the functional domain. Because, you can see that convolution operation the functional domain that becomes equivalent to multiplication operation in the transpose domain; when you considered the Fourier transform.

So, this is the particular expressions as these are stage by which we can understand; that the relation is that if I consider a function and if the impulse response of the system is h(t); then convolution of impulse response with that function is equivalent to the product of Fourier transform of this function and Fourier transform of this impulse function. This property is called convolution multiplication property of Fourier transforms.

(Refer Slide Time: 14:19)



Let us consider how this property is reflected using discrete Fourier transform. In the discrete domain the convolution operation; linear convolution operation can be represented in this form instead of integrations now we have summations and these are shifted impulse responses; that is unit impulse responses along the corresponding functional domain at integral points.

And the linear combination of all those shifted values will give you the convolutions as you see that coefficients whose linear combination comes from the functional value itself. So, when you perform the linear convolution; the thing is that, we assume both of the sequence and also the impulse response they are of infinite length. But it happens so, we are handling with finite sequence and discrete Fourier transform is applied for a finite sequence.

So, how do you consider this definitions, how do you we can modify this definition because it is not necessarily the corresponding functional domain where the function is not defined in a finite sequence; it is not necessarily they have to be set to 0. If you set them to 0 it becomes equivalent to linear convolution, but you can consider some periodic extension as we did for discrete Fourier transform definition itself. So, periodic convolution of two finite sequences is defined as convolution between two finite sequences with periodic extension.

Same linear convolution and it can be observed that if they have same period that the periodic sequence also will be of same period. So, with this definition; with this property we can define a circular convolution. So, it is sufficient if we compute the periodic convolutions for a single period only, it need not compute in the whole functional domain itself.

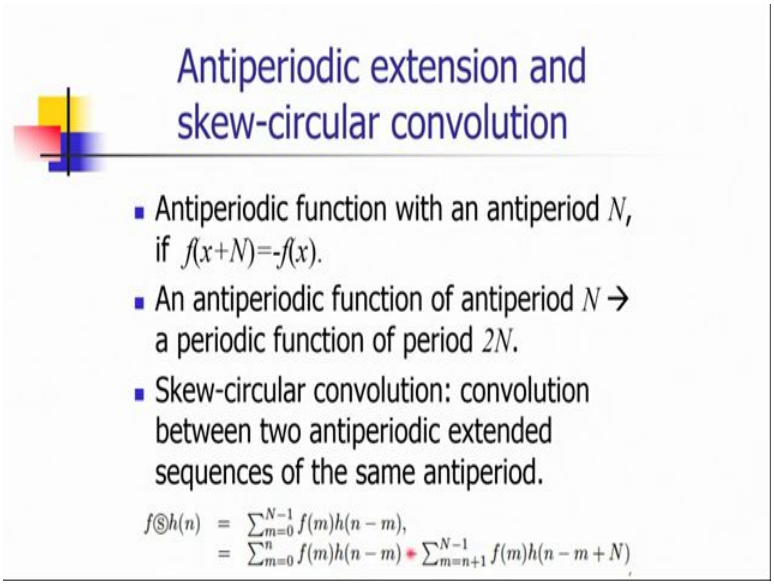
$$f \circledast h = \sum_{m=0}^{N-1} f(m)h(n-m)$$

$$= \sum_{m=0}^{n} f(m)h(n-m) + \sum_{m=n+1}^{N-1} f(m)h(n-m+N)$$

So, in a circular convolution which is as I mentioned, it is a periodic extension and we can compute only with the interval of from 0 to N-1. And if I apply this periodic periodicity definition; then this can be broken into the two parts as shown above. This is the definition of a circular convolution. The interesting part is that convolution multiplication property for DFT holds for the circular convolution.

So if I consider the impulse response of a system and functional finite sequence both should be of same length; take the discrete Fourier transform and take the product point wise at the corresponding coefficient wise take the product. Then, you will get also the transform coefficients of the sequence what would have been the output of the system if they are all periodic extensions.

(Refer Slide Time: 17:36)



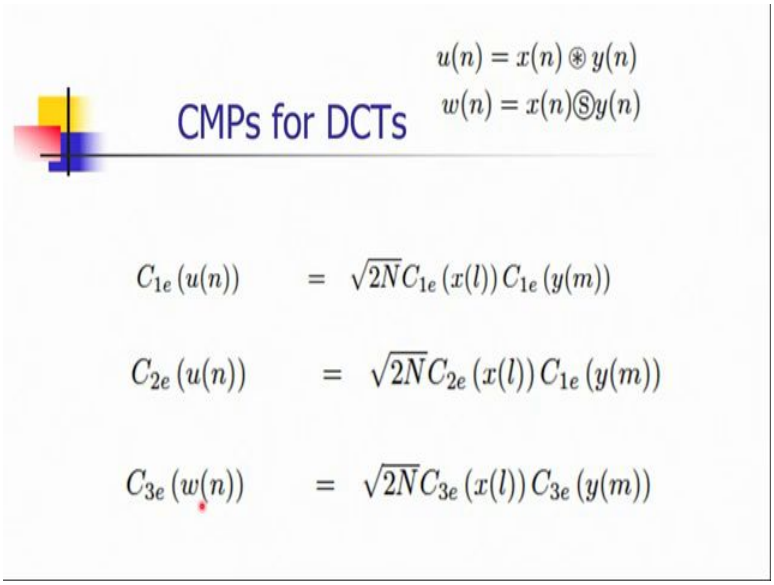
So, this is what is your is the property in the discrete Fourier transform domain. So, what about; there is another kind of convolution with antiperiodic extension. Like periodic extension, we can have antiperiodic extension; in the antiperiodic extension first we perform the this antiperiodic extension over the functional domain that is with an antiperiod N.

$f(x+N) = -f(x)$

This is what it is defined antiperiodic function and if I do this antiperiodic extension; it is it is also a periodic function of period N that is interesting.I I antiperiodic function does not mean it is; it is not periodic. It is actually periodic, but the periodicity is now doubled you can check this things; you can make an antiperiodic extension and we will see there is a periodicity of twice the number of samples of the original sequence.

So, skew circular convolution is defined with respect to this periodic values or we can say this antiperiodic extensions. Because we will be again observing the convolved output only in the observation window of the original sequence. So, a skew circular convolution is defined in this form; you take the convolutions again from the linear convolution definition itself, but you apply the properties of antiperiodic extension; then you will get this expression.

(Refer Slide Time: 19:18)

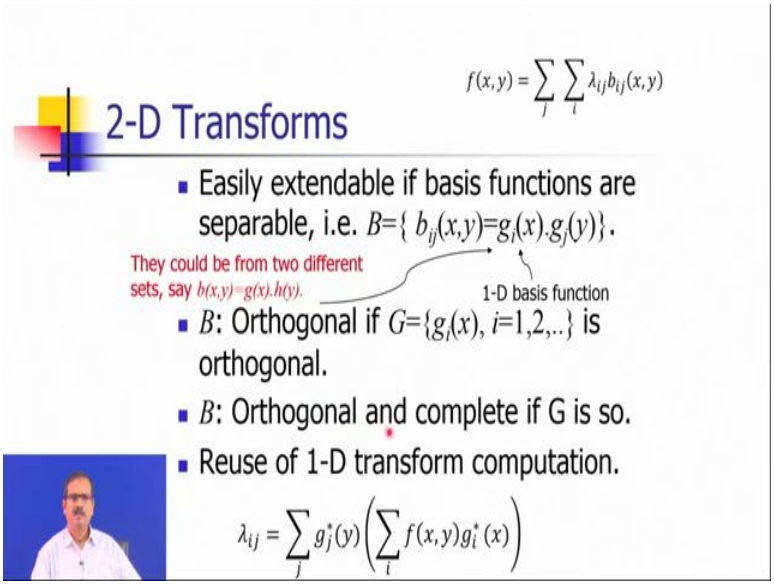


So, using the circular convolution and skew circular convolution; we can find that there are different convolution multiplication properties those hold also in DCTs and DSTs. I will show you some of them for DCTs. For example, you take this case that you have two functions; one of them you can consider as the impulse response say y and both of them of same period; of same length. So, if you take the type I DCT of this one and type I DCT of this one; then if I multiply and no that is that should be multiplied by this factor because of the definition of DCTs what we have in this lecture following that this multiplication factor would there.

And then you get your result in the transform domain itself; that is the type I DCT of the circularly convolved output of these two sequences. Similarly, a type II DCT of this one and type I DCT of impulse response will give you a type II DCT of the corresponding convolved result. You should note that number of samples depend upon the corresponding type of DCT what you are applying. Because finally, there should be of same periodicity; so in this case we will have N+1 samples and there are N samples because N sample define a DCT of 2N period and N+1 samples define it type I DCT of 2N period.

So, that you need to be careful while applying this particular properties and type III DCTs have these interesting property; where actually you can find the output is this, this properties applied for skew circular convolution.

(Refer Slide Time: 21:20)



So, whatever we have discussed in one dimensional transform; this can be easily extended for two dimensional transforms this discussion. If I consider our basis function in two dimension as a certain particular property; which is separability property; so they are separable if I can I write this basis function into this two form; $B = \{b_{ij}(x,y) = g_i(x).g_j(y)\}$

that means, you can write it as a product of 2 basis functions as shown above; they are all independent of independently they can be computed using a particular variable. So, this is an 1 D basis function say product of two one dimensional basis function.

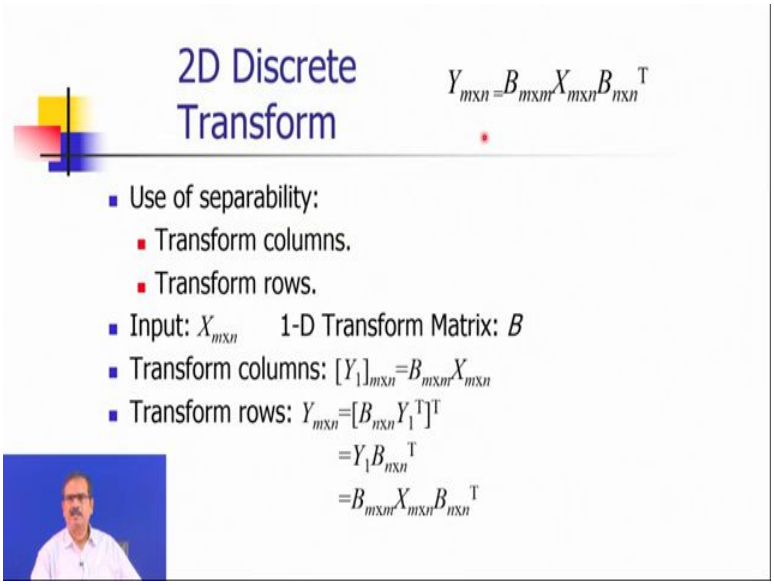
If both of them are orthogonal; then you can see that this set of basis functions will be also orthogonal and then we can reuse this one dimensional transform computation and we can express them in the following equation

$$\lambda_{ij} = \sum_j g_j^*(y)(\sum_i f(x,y)g_i^*(x))$$

So, first we are computing the transforms with respect to the x; by changing the values or sequence with respect to variation over x.

And then we are considering the transform of say with respect to the value of y, we will see that how this computation is reflected in terms of matrix operation. So, you should note here though in this slide; we have used the same notations for this two functions; they could be separate, only thing is that both of them needs to be orthogonal to keep the orthogonality property of this basis functions.

(Refer Slide Time: 23:11)



So, 2D discrete transform can be easily computed by using this separability property. So, let us consider the computation being this way say you can transform columns and then you can transform rows. Suppose, you have a 1 dimensional transform matrix B and there is an input. your input is a corresponding input which is given in terms of $m \times n$ matrix ,that is your data.

So,first I can transform columns of this input data; that is the image in this case $m \times n$, some image block. So, I will be transforming them columns; so, we know that dimension of each

column is m So, that is why we are use using the corresponding transformation matrix which deals with m dimensional vectors; which is represented in this form and then each column is now is transformed into another m columns. So, you are doing it for n such columns. So, you will find n dimensional vectors transform columns and there are n columns; so this is a matrix you will get.

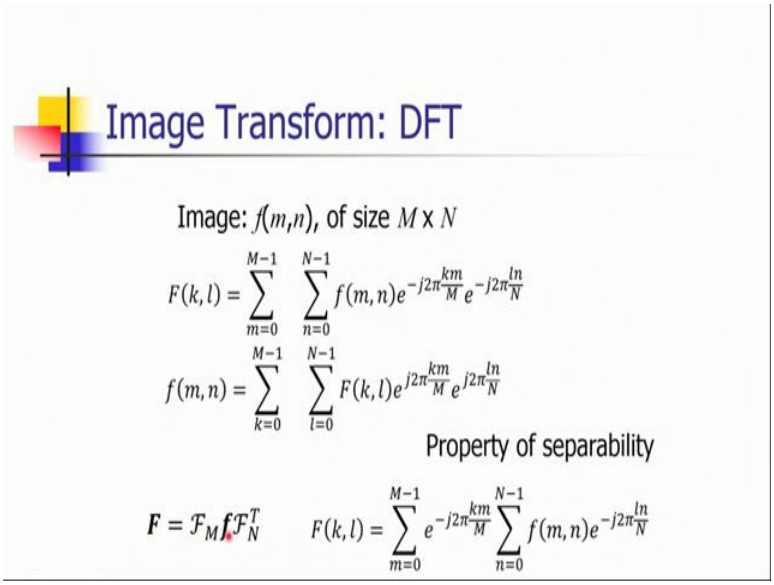
After that what you can do ,you can now transform the rows of this matrix which means now you have to take the transpose and then perform the $n \times n$. Since all rows are n dimensional, so you have to use the n-point transformation matrix here and then you can get the final no transforms of the two dimensional image of $m \times n$ size.

So, if I expand Y_1 I can write the whole operations in the following composite form.

$$Y_{m\times n} = [B_{n\times n}(Y_1)^T]^T = Y_1(B_{n\times n})^T = B_{m\times m}X_{m\times n}(B_{n\times n})^T$$

So, the whole operations can be described in this particular form and that gives me the 2 dimensional discrete form. So, this is how a 2 dimensional discrete form can be represented given the 2 dimensional input image we can use the corresponding transformation matrix towards its right and towards its left in this way and we can get the corresponding transformed image in the transformed domain itself.

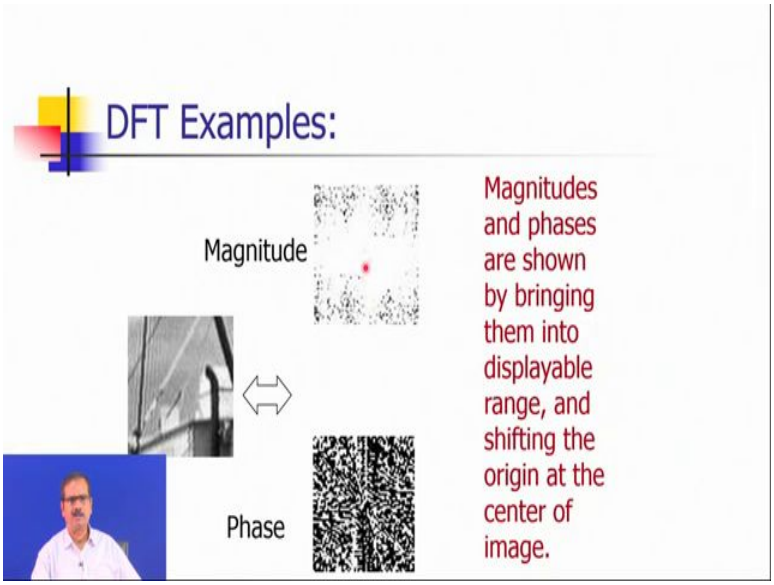
(Refer Slide Time: 25:39)



So, typical examples could be say discrete Fourier transform; we can also express this transforms using the summation operations because they are the basis functions here or the basis vectors here; they are separable, they are expressed in this form. And you can see that it is a simple extension of the discrete Fourier transform of what you had in the in your in the 1 dimensional case. And using property of separability, once again we can simplify this computation and when we express in terms of matrix multiplication; we have already defined the matrix representation of the transformation matrix of DFT.

And we consider this is a $m \times m$ ransformation matrix and this is a $n \times n$ matrix; we are just denoting it by simple M showing it as m point discrete Fourier transformation matrix. So, this will give you the corresponding transformation of the image f.

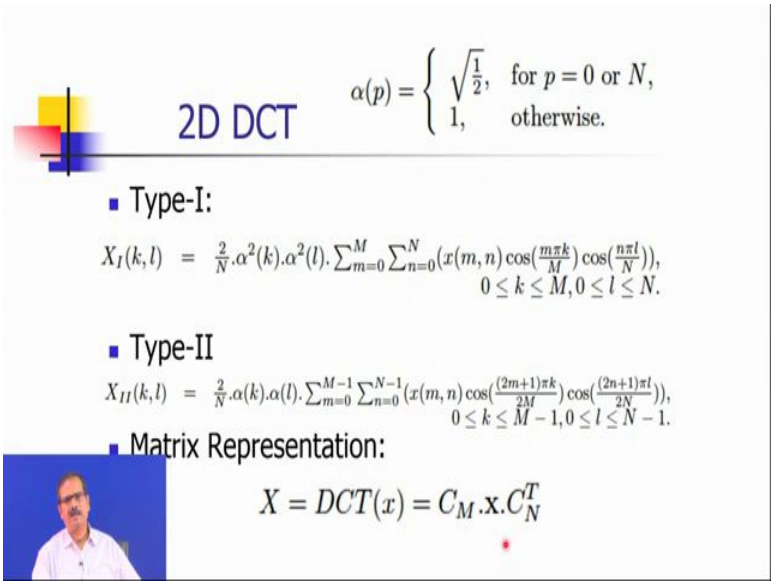
(Refer Slide Time: 26:47)



So, one typical example it is shown here that now given this image; we can perform Fourier transform and it will give me since it is in the complex quantity every transform coefficient in the discrete Fourier transform is a complex element. So, it has its magnitude and phase at every point; so you get two components of this transformation. And in this particular image you should note that if I apply Fourier transform, coefficient values would be very large and it is very difficult to make them display.

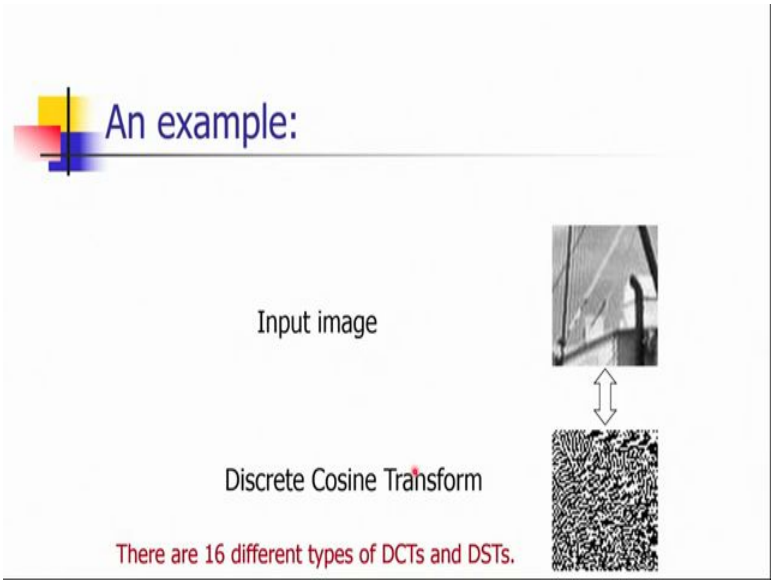
So, we have made them display well and also we have shifted the origin of the transformation space.

(Refer Slide Time: 27:33)



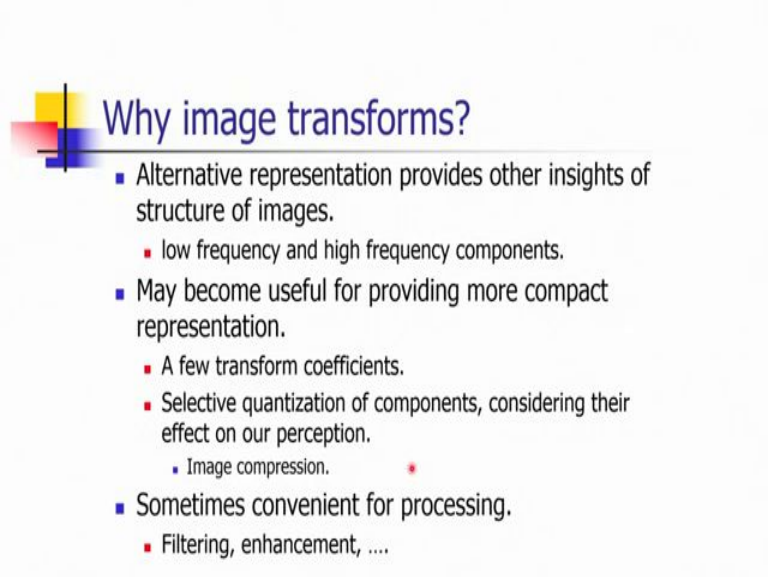
So similar way like discrete Fourier transform, we can define also discrete two dimensional discrete cosine transform. These are same simple extensions what we had in 1 dimension and in the matrix representation; we have the similar representation what we discussed for when we are extending a 1 dimensional transform to a 2 dimensional transform.

(Refer Slide Time: 28:00)



This is an example of a DCT; so if given this input image, you have this discrete cosine transform. In this case also you have scaling of the coefficient values and there are as I have mentioned there could be 16 different types of DCT and DSTs this is just one typical example which is type II even DCT that is shown here.

(Refer Slide Time: 28:27)



So, I will conclude this lecture by mentioning that why do we require image transforms? As you can see image transform, give an alternative representation of image instead of representing the image in the functional domain itself, we are representing it in a different domain; it gives a different insight of structure of images.

And for example, if I consider the frequency domain representations using Fourier transforms, we get low frequency and high frequency components. So, it may become useful for providing more compact representation; you can use only a few transform coefficients to get a very close approximation of the functional representation or you can perform selective quantization of components, considering their effect on our perception.

And those are used in the algorithms for image compressions, even video compressions. And sometimes many processing becomes convenient when we use this transform coefficients; like

we have already discussed about filtering, there are operations like enhancement, restoration etc many other operations; where this transforms are useful. So, with this I end my lecture on image transforms.

Thank you very much for your patience and listening to my lecture.